

Responsible Software Development with LLMs

Presented by



<https://pesoproject.org> <https://rapids.lbl.gov>

Members of



*Consortium for the Advancement
of Scientific Software*

<https://cass.community>

With prior support from



Anshu Dubey (she/her)
Argonne National Laboratory

Better Software for Reproducible Science tutorial @ The International Conference for High-Performance Computing, Networking, Storage, and Analysis (SC25) conference

Contributors: Anshu Dubey (ANL), David E. Bernholdt (ORNL)



See slide 2 for
license details

License, Citation and Acknowledgements

License and Citation



- This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0).
- **The requested citation the overall tutorial is:** David E. Bernholdt and Anshu Dubey, Better Software for Reproducible Science tutorial, in The International Conference for High-Performance Computing, Networking, Storage, and Analysis (SC25), St. Louis, Missouri, 2025. DOI: [10.6084/m9.figshare.30394186](https://doi.org/10.6084/m9.figshare.30394186).
- Individual modules may be cited as *Speaker, Module Title, in Tutorial Title, ...*

Acknowledgements

- This work was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Next-Generation Scientific Software Technologies (NGSST) and Scientific Discovery through Advanced Computing (SciDAC) programs.
- This work was supported by the U.S. Department of Energy Office of Science, Office of Advanced Scientific Computing Research (ASCR), and by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration.
- This work was performed in part at the Argonne National Laboratory, which is managed by UChicago Argonne, LLC for the U.S. Department of Energy under Contract No. DE-AC02-06CH11357.
- This work was performed in part at the Lawrence Livermore National Laboratory, which is managed by Lawrence Livermore National Security, LLC for the U.S. Department of Energy under Contract No. DE-AC52-07NA27344.
- This work was performed in part at the Los Alamos National Laboratory, which is managed by Triad National Security, LLC for the U.S. Department of Energy under Contract No.89233218CNA000001
- This work was performed in part at the Oak Ridge National Laboratory, which is managed by UT-Battelle, LLC for the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.
- This work was performed in part at Sandia National Laboratories. Sandia National Laboratories is a multi-mission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

Introduction

- Investing some thought in design of software makes it possible to maintain, reuse and extend it
- Even if some research software begins its life as a one-off use case, it often gets reused
 - Without proper design it is likely to accrete features haphazardly and become a monstrosity
 - Acquires a lot of technical debt in the process
 - Many projects have had this happen
 - Most end up with a hard reset and start over again
- Arrival of LLMs and their usefulness in increasing productivity has made this topic even more relevant
- Rigorous verification is absolutely essential for using LLMs in any coding task.
- In this module we will cover general design principles with test-driven development, and verification, and apply them to coding with LLM

Designing Software – High Level Phases

Requirements gathering

- ☐ Features and capabilities
- ☐ Constraints
- ☐ Limitations
- ☐ Target users
- ☐ Other

Decomposition

- ☐ Understand design space
- ☐ Decompose into high level components
- ☐ Bin components into types

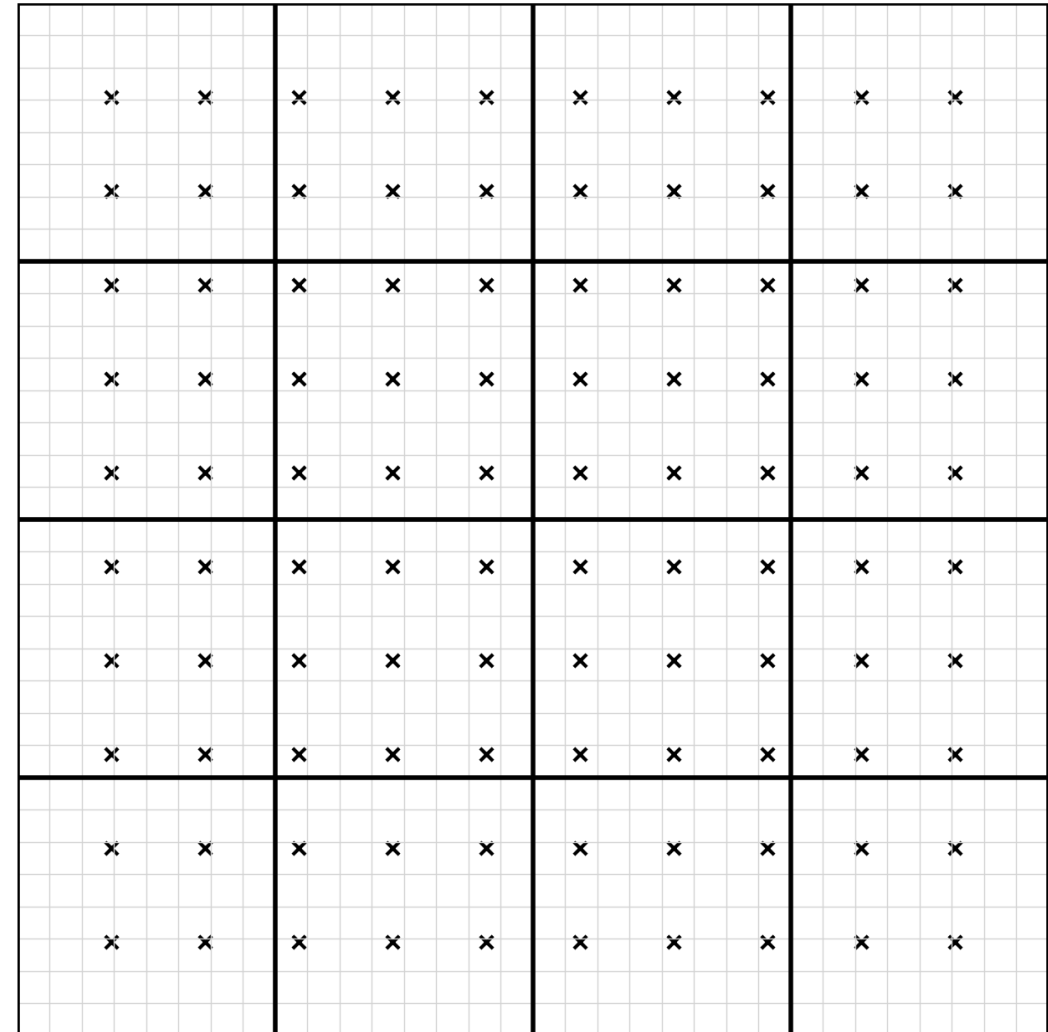
Connectivity

- ☐ Understand component hierarchy
- ☐ Figure out connectivity among components
- ☐ Articulate dependencies

Example

- Many applications combine mesh and particles
 - Particles have positions associated with them
 - They interchange physical quantities with the mesh
 - They change positions based on interactions
- We will build a uniformly discretized 2D cartesian mesh and initialize with a lattice of particles
 - We will divide the mesh into blocks
 - We will associate each particle with a single block
 - We will move particles to different locations which may change the block they are associated with
 - We will need to handle particles that leave the domain

(64.0,64.0)



(0,0)

Requirements Gathering

- Components we need
 - Mesh generator – with mesh divided into blocks
 - Particles generator – in a 2D lattice
 - For each particle ability to locate the block on which the particle resides
 - Making the particles change position
- To verify that we are doing it correctly we need to make sure that
 - The mesh is generated correctly
 - The particles are initialized in a lattice and associated with blocks correctly
 - After moving the particles change their associations correctly

Decomposition

This is a small design space

❑ Several requirements can directly map to components

- ❑ Driver
- ❑ Mesh initialization
 - ❑ Data structure
 - ❑ Division into blocks
- ❑ Particle initialization
 - ❑ Position
 - ❑ Association with a mesh block
- ❑ Particle mover
- ❑ Tests

Binning components

❑ Infrastructure Components

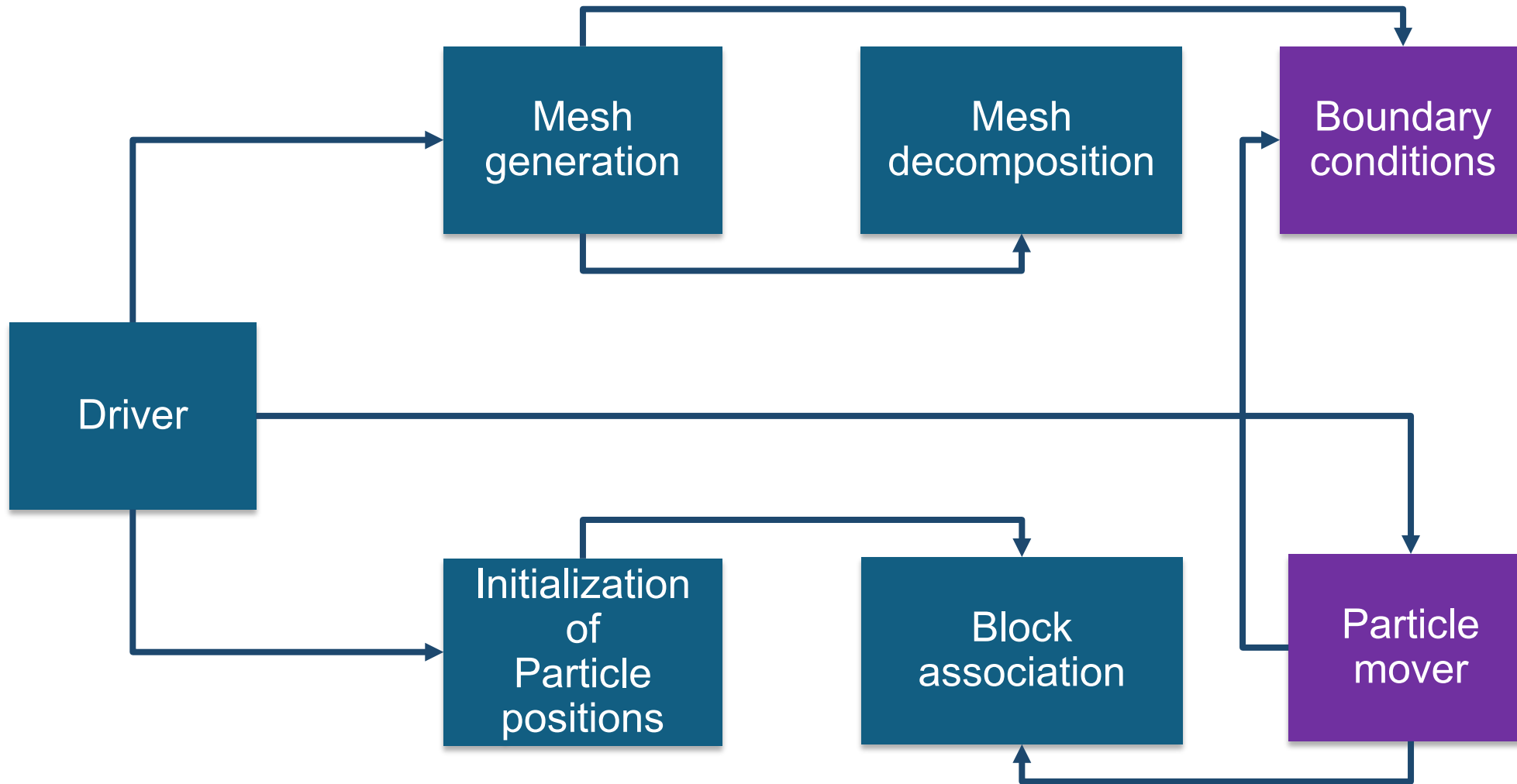
- ❑ Driver
- ❑ Initialization
 - ❑ Mesh with division into blocks
 - ❑ Particles position
- ❑ Association with a mesh block

❑ Components that are specific to one instance

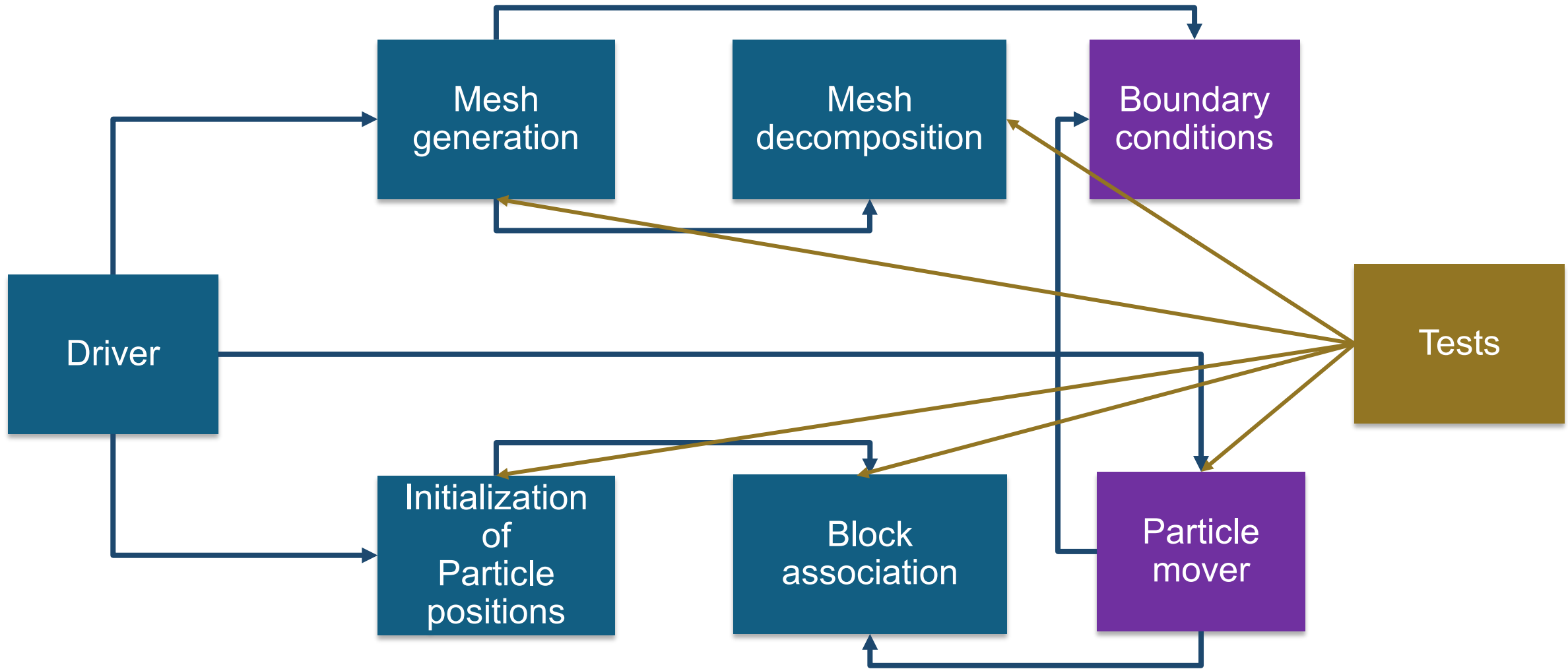
- ❑ Boundary conditions
- ❑ Particle mover

❑ Components for verification

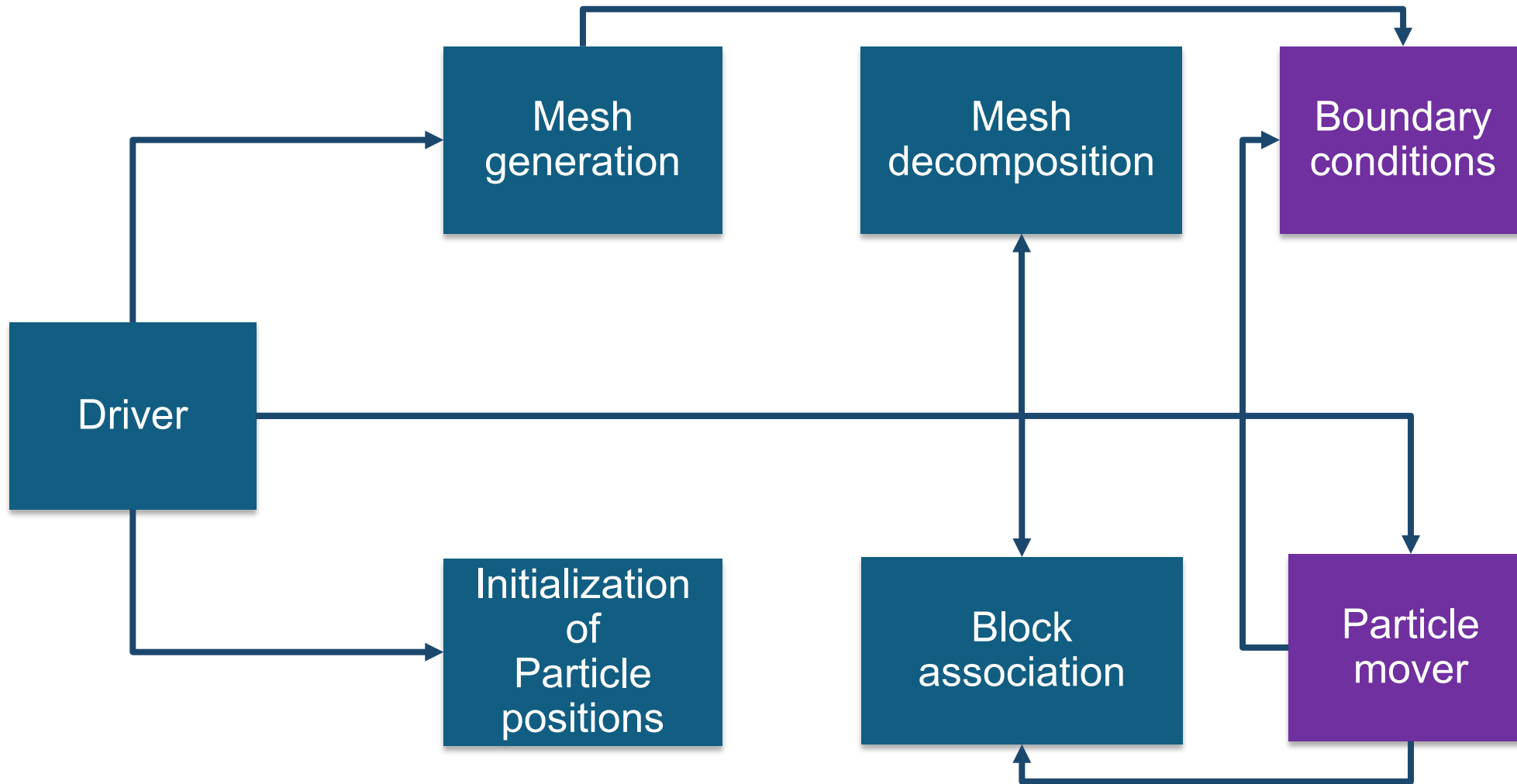
Connectivity



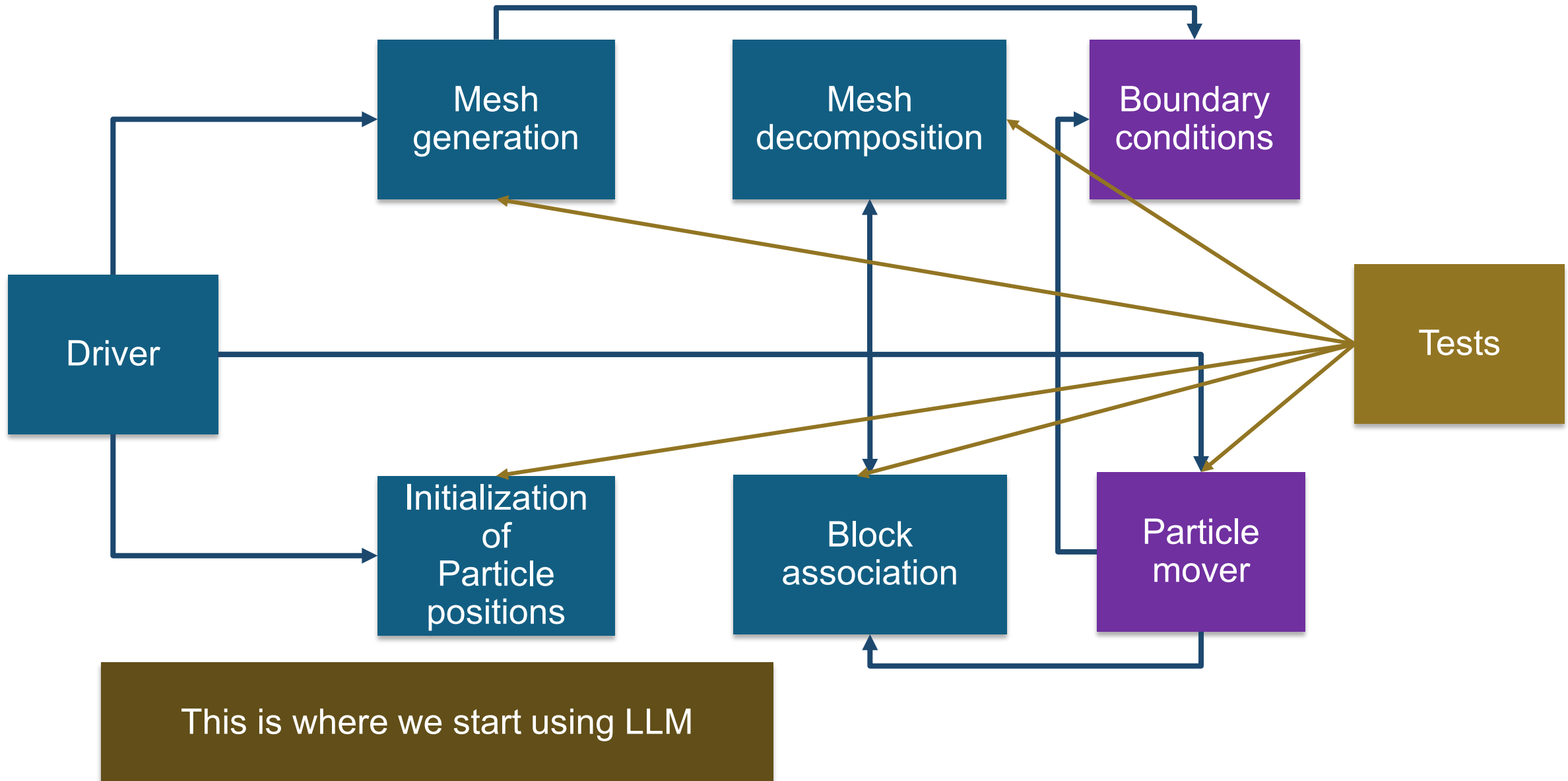
Connectivity



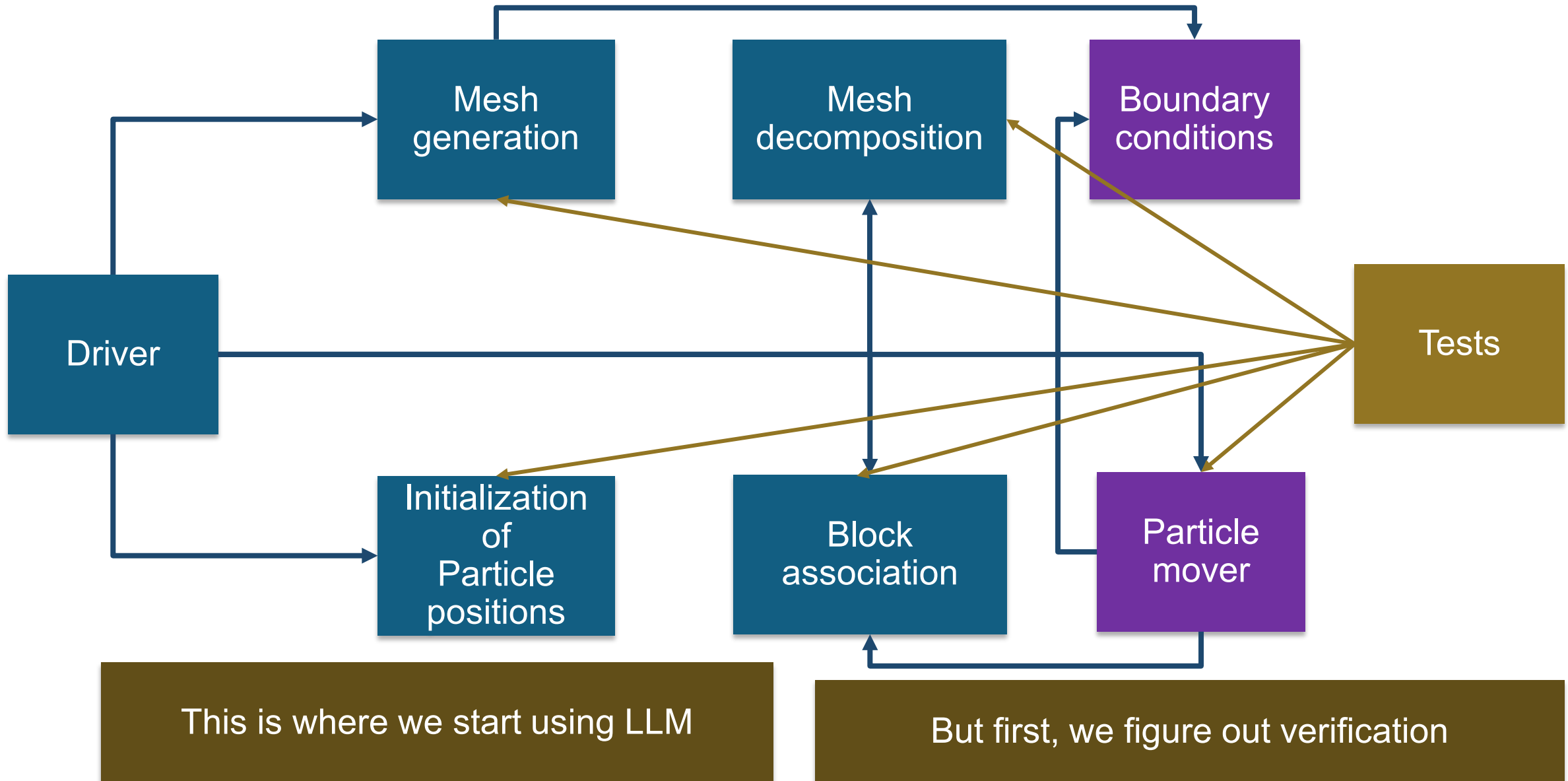
Connectivity



Connectivity



Connectivity



What is Testing

Whenever you write a code you are doing it

- When you compile it, you are testing for defects in syntax
- When you run it for the first time you are testing for correctness
- When you add any code and run it again, you are testing it again
- When you break down your development into smaller chunks you test each chunk, then you combine the chunks, and you test again.

What is Testing

Whenever you write a code you are doing it

- When you compile it, you are testing for defects in syntax
- When you run it for the first time you are testing for correctness
- When you add any code and run it again, you are testing it again
- When you break down your development into smaller chunks you test each chunk, then you combine the chunks, and you test again.

Testing is an integral part of code development

What is Testing

Whenever you write a code you are doing it

- When you compile it, you are testing for defects in syntax
- When you run it for the first time you are testing for correctness
- When you add any code and run it again, you are testing it again
- When you break down your development into smaller chunks you test each chunk, then you combine the chunks, and you test again.

Testing is an integral part of code development

So, what is the whole fuss about testing?

What is Testing

Whenever you write a code you are doing it

- When you compile it, you are testing for defects in syntax
- When you run it for the first time you are testing for correctness
- When you add any code and run it again, you are testing it again
- When you break down your development into smaller chunks you test each chunk, then you combine the chunks, and you test again.

Testing is an integral part of code development

So, what is the whole fuss about testing?

Formalization of the process has been intimidating to people because they think of writing tests as an overhead

How to Think About Building Tests

You start by thinking about what is the correct behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

How to Think About Building Tests

You start by thinking about what is the correct behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

You also think about what would be wrong behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

How to Think About Building Tests

You start by thinking about what is the correct behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

You also think about what would be wrong behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

Let us work through an example ...

- You want a large prime number for encryption
- As a part of the development, you first write a function that checks if a given number is prime

Correct behavior: input 13 returns true,
input 15 returns false
Incorrect behavior: input 15 returns true

How to Think About Building Tests

You start by thinking about what is the correct behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

You also think about what would be wrong behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

Let us work through an example ...

- You want a large prime number for encryption
- As a part of the development, you first write a function that checks if a given number is prime

Correct behavior: input 13 returns true,
input 15 returns false
Incorrect behavior: input 15 returns true

Here are all the ingredients
for building a test !!

How to Think About Building Tests

You start by thinking about what is the correct behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

You also think about what would be wrong behavior

Next you think about how you are going to be able to tell whether the code is exhibiting correct behavior

- You write a “main” that reads in a number, calls the functions and prints true or false
- You can automate it by including a series of known primes and non-primes and their corresponding true or false values
- This is your “unit test” for the function

Let us work through an example ...

- You want a large prime number for encryption
- As a part of the development, you first write a function that checks if a given number is prime

Correct behavior: input 13 returns true,
input 15 returns false
Incorrect behavior: input 15 returns true

Here are all the ingredients
for building a test !!

How to Think About Building Tests

Next you write a function to get to a large prime for encryption

Then you wish to confirm that it is a large enough prime

So, you write another unit test that counts the number of digits in the prime

How to Think About Building Tests

Next you write a function to get to a large prime for encryption

Then you wish to confirm that it is a large enough prime

So, you write another unit test that counts the number of digits in the prime

Finally, you want to verify that it meets your encryption needs

You integrate your new function with your encryption software

The encryption software is likely to have a way to verify that the cipher can only be translated with the right key

How to Think About Building Tests

Next you write a function to get to a large prime for encryption

Then you wish to confirm that it is a large enough prime

So, you write another unit test that counts the number of digits in the prime

Finally, you want to verify that it meets your encryption needs

You integrate your new function with your encryption software

The encryption software is likely to have a way to verify that the cipher can only be translated with the right key

- Now you have a more complex test that involves several correctly working components
- This is your “integration test”

Types of Tests

Well known tests for enterprise software

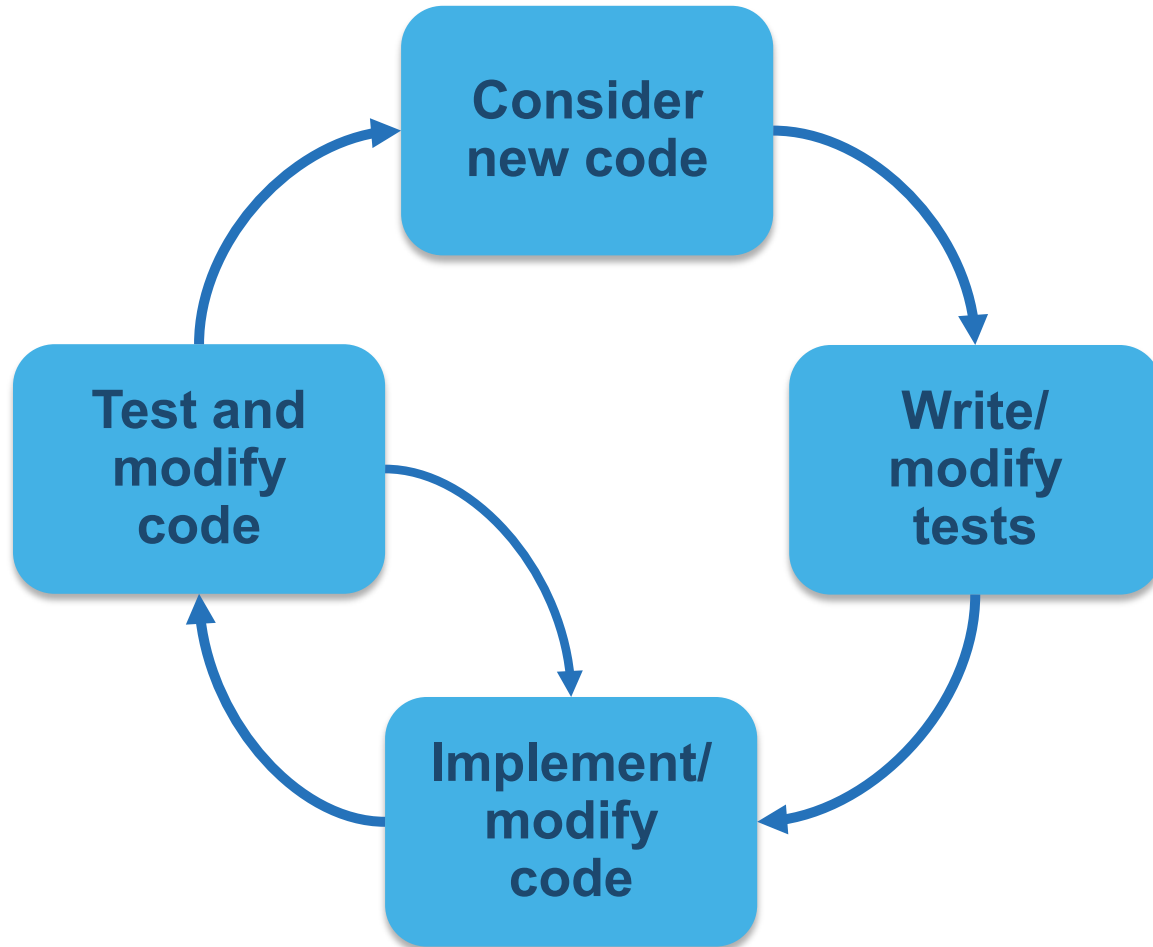
- Unit tests – verify a single function, extremely quick to run
- Integration tests – verify functions working together
- System tests – verify functionality of the entire software
- Acceptance tests – verify that the client needs are met
- Regression tests – verify that there is no degradation in code capabilities

Types of Tests

Additional types of tests needed for research software

- Composite unit tests – are tests for specific functionalities and/or capabilities
- Granular tests – are integration tests at various granularities verifying correct behavior of interoperating functional units
- Restart tests – verify that a run can restart transparently from a checkpointed state
- Performance tests – apply to high-performance computing codes, verify that there is no performance loss

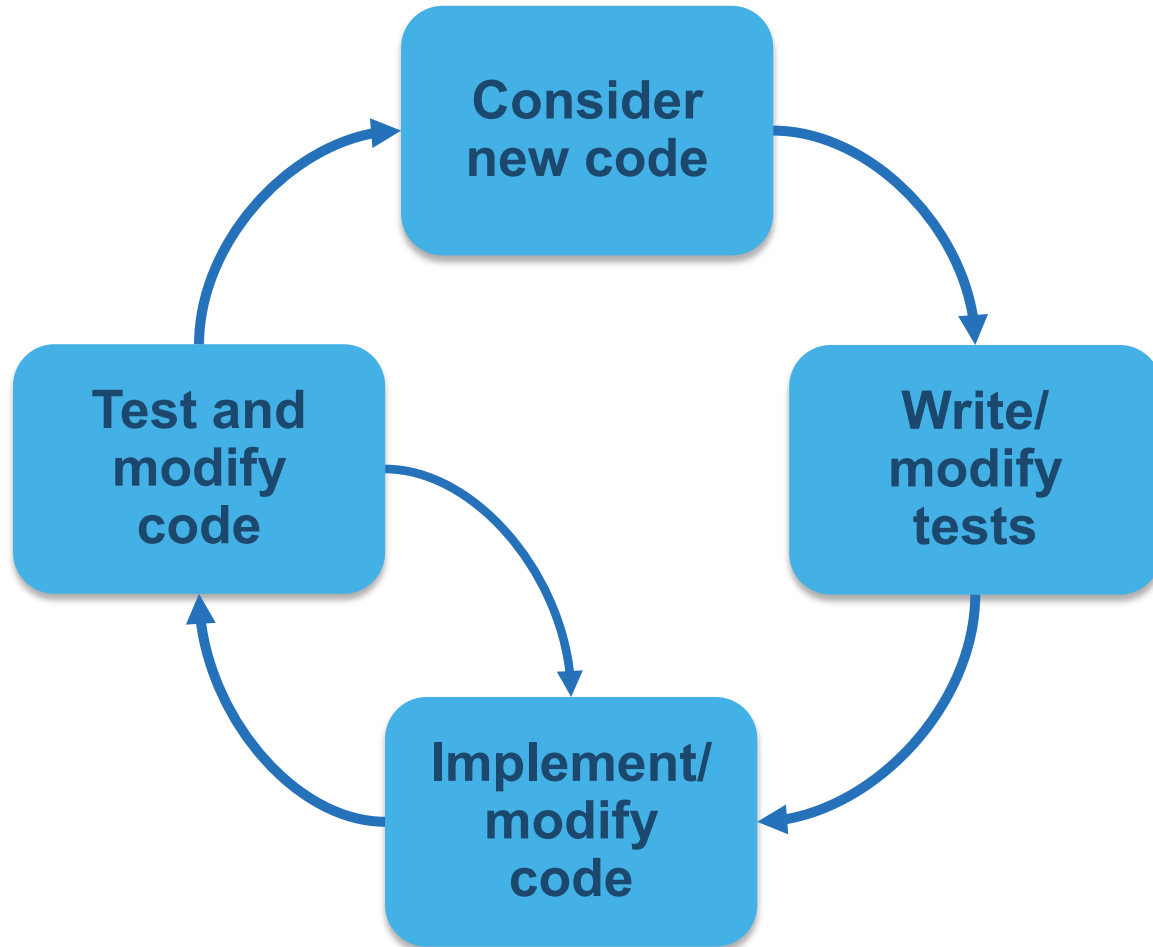
Test Driven Development



- Documented specifications and requirements of the code
- Ensures that thought is given to what it means for the program to be correct, rather than just what the program should do
- More efficient development cycle
- Much less debugging
- Requires:
 - Care in writing tests
 - Frequent running of tests
 - Wide adoption by development team

We will do TDD for our mesh/particle problem

Test Driven Development



- Documented specifications and requirements of the code
- Ensures that thought is given to what it means for the program to be correct, rather than just what the program should do
- More efficient development cycle
- Much less debugging
- Requires:
 - Care in writing tests
 - Frequent running of tests
 - Wide adoption by development team

We will do TDD for our mesh/particle problem

Now we are really ready to use LLM

Using LLM

In the hands-on info, we had provided a tutorial for CodeScribe— getting and using it

- If you don't have CodeScribe you can still use web-interface
 - For this example CodeScribe is not really needed.
- In every step of the development we will explain the requirement in the prompt, ensure that the model has not misinterpreted any of the requirements, and then ask it to code.
- For every function generated we will also ask for a test – in this case our tests are going to rely upon visual inspection by the human-in-the-loop – you.
- We will provide a set of prompts as reference, but you should use your own.
- For CodeScribe the prompts are appended in the toml file, in direct interaction you have to provide them one after another.

Code Generation Steps

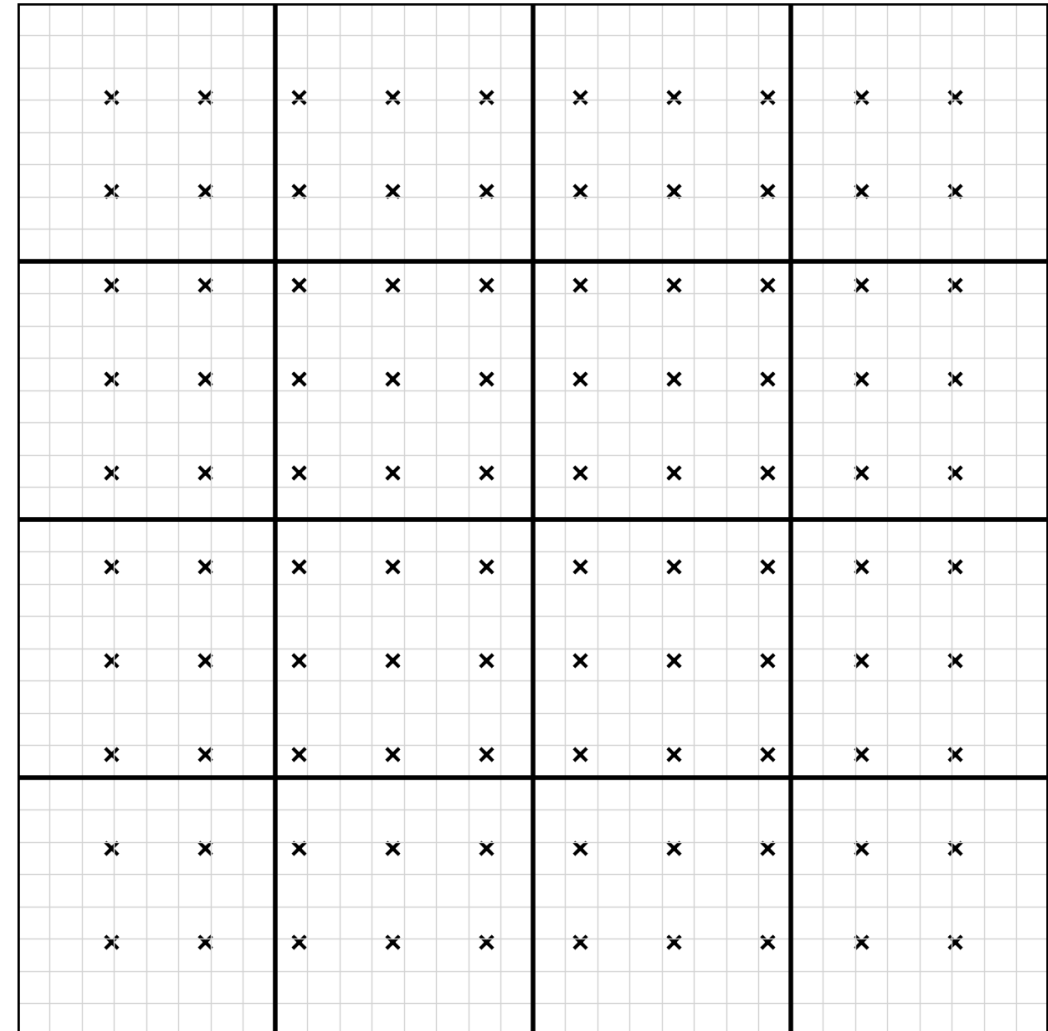
If you wish to use your own methodology, feel free to do so

- Step 1 – Generate the mesh
 - Bounds $\langle 0,0 \rangle$ to $\langle 64,64 \rangle$
 - Divided into 16 blocks, numbered in lexicographic order
 - Keep track of bounds for each block
- Test needs
 - Function to print all the meta information about the mesh – domain bounds, blocks, bounds for each block and block identity
 - Human-in-the-loop checks the printed values against the figure on the right.

Code Generation Steps

(64.0,64.0)

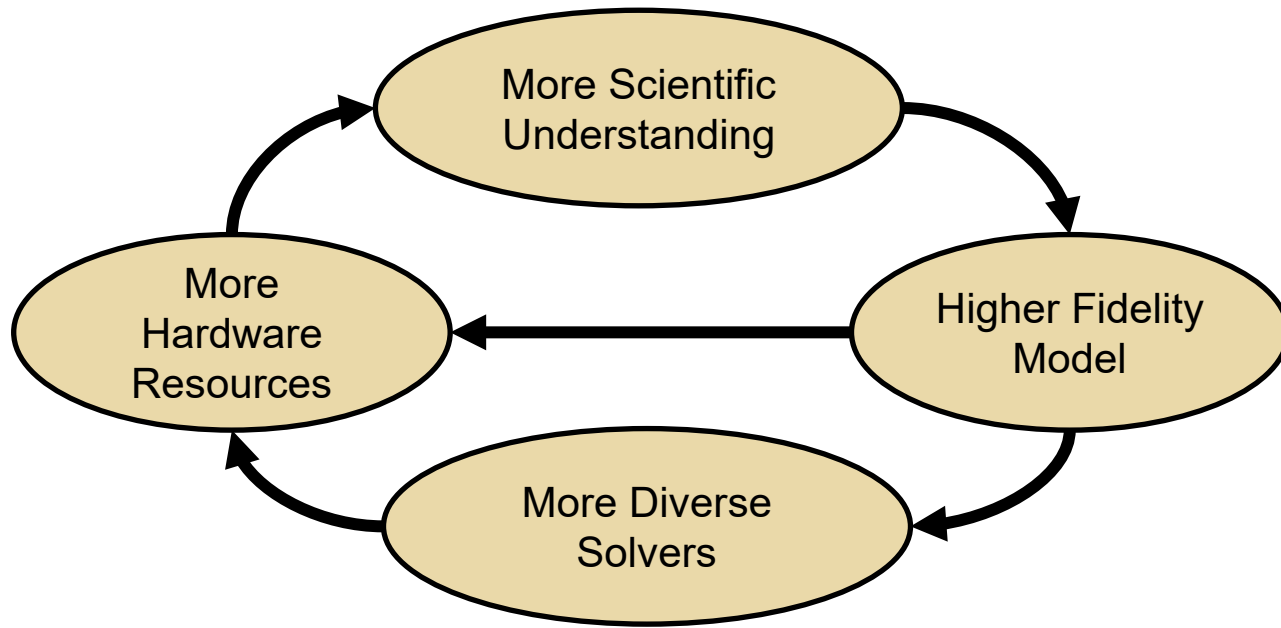
- Step 2 – Place particles on the mesh
 - In this figure the lattice is 10x10
 - Each particle has 3 values connected to it. $\langle x,y \rangle$ position coordinates, and the block that it is associated with
 - Needs a function to find the block that overlaps with the particle position
- Test needs
 - To test the function to find the right block user is prompted to give a position coordinate, the tester prints the block number, and the user verifies that it is correct
 - To test that particles are initialized correctly process the entire list of particles and confirm that they are associated with the right block



(0,0)

Switch to terminal

Advanced Topic – Research Software Challenges in Design and Verification



- Many parts of the model and software system can be under research
- Requirements change throughout the lifecycle as knowledge grows
- Verification complicated by floating point representation
- Real world is messy

SOLID Principles Pose Some Difficulties

- ❑ Single responsibility
 - ❑ Class/method/function should do only one thing
- ❑ Open/closed
 - ❑ Open for extension, close for modification
- ❑ Liskov substitution
 - ❑ Implementations of an interface should give same result
- ❑ Interface segregation
 - ❑ Client should not have to use methods it does not need
- ❑ Dependency inversion
 - ❑ High level modules should not depend on low level modules, only on abstractions

- ❑ Function calls have overheads
 - ❑ Performance matters – quick turnaround of results desirable
- ❑ New insights may cause modification
 - ❑ May lead to unmaintainable code duplication
- ❑ It is not always possible to eliminate lateral interactions
- ❑ Not always possible

Additional Considerations for Research Software

Considerations

- ❑ Multidisciplinary
 - ❑ Many facets of knowledge
 - ❑ To know everything is not feasible
- ❑ Two types of code components
 - ❑ Infrastructure (mesh/IO/runtime ...)
 - ❑ Science models (numerical methods)
- ❑ Codes grow
 - ❑ New ideas => new features
 - ❑ Code reuse by others

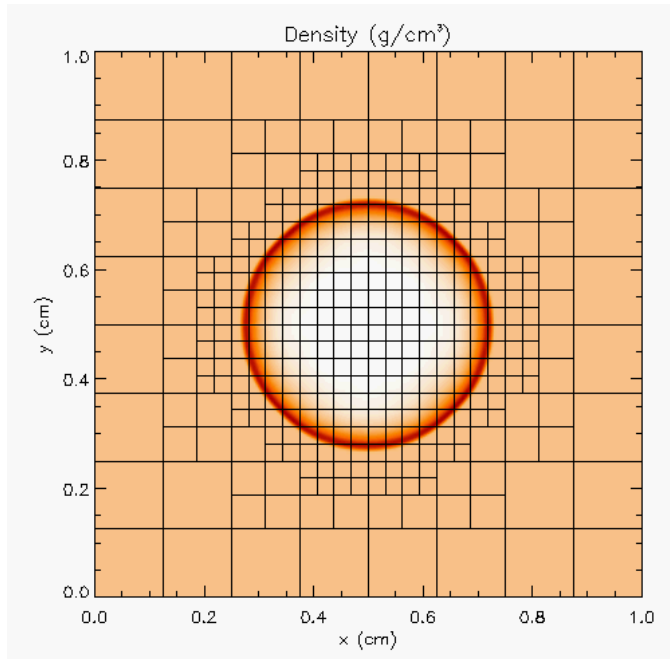
Design Implications

- ❑ Separation of Concerns
 - ❑ Shield developers from unnecessary complexities
- ❑ Work with different lifecycles
 - ❑ Long-lasting vs quick changing
 - ❑ Logically vs mathematically complex
- ❑ Extensibility built in
 - ❑ Ease of adding new capabilities
 - ❑ Customizing existing capabilities

More Complex Application Design – Sedov Blast Wave

Description

High pressure at the center cause a shock to moves out in a circle. High resolution is needed only at and near the shock



Requirements

- Adaptive mesh refinement
 - Easiest with finite volume methods
- Driver
- I/O
- Initial condition
- Boundary condition
- Shock Hydrodynamics
- Ideal gas equation of state
- Method of verification

Deeper Dive into Requirements

- Adaptive mesh refinement → divide domain into blocks
 - Blocks need halos to be filled with values from neighbors or boundary conditions
 - At fine-coarse boundaries there is interpolation and restriction
 - Blocks are dynamic, go in and out of existence
 - Conservation needs reconciliation at fine-coarse boundaries
- Shock hydrodynamics
 - Solver for Euler's equations at discontinuities
 - EOS provides closure
 - Riemann solver
 - Halo cells are fine-coarse boundaries need EOS after interpolation
- Method of verification
 - An indirect way of checking – shock distance traveled can be computed analytically

Components

Binned Components

☐ Unchanging or slow changing infrastructure

- ☐ Mesh
- ☐ I/O
- ☐ Driver
- ☐ Comparison utility

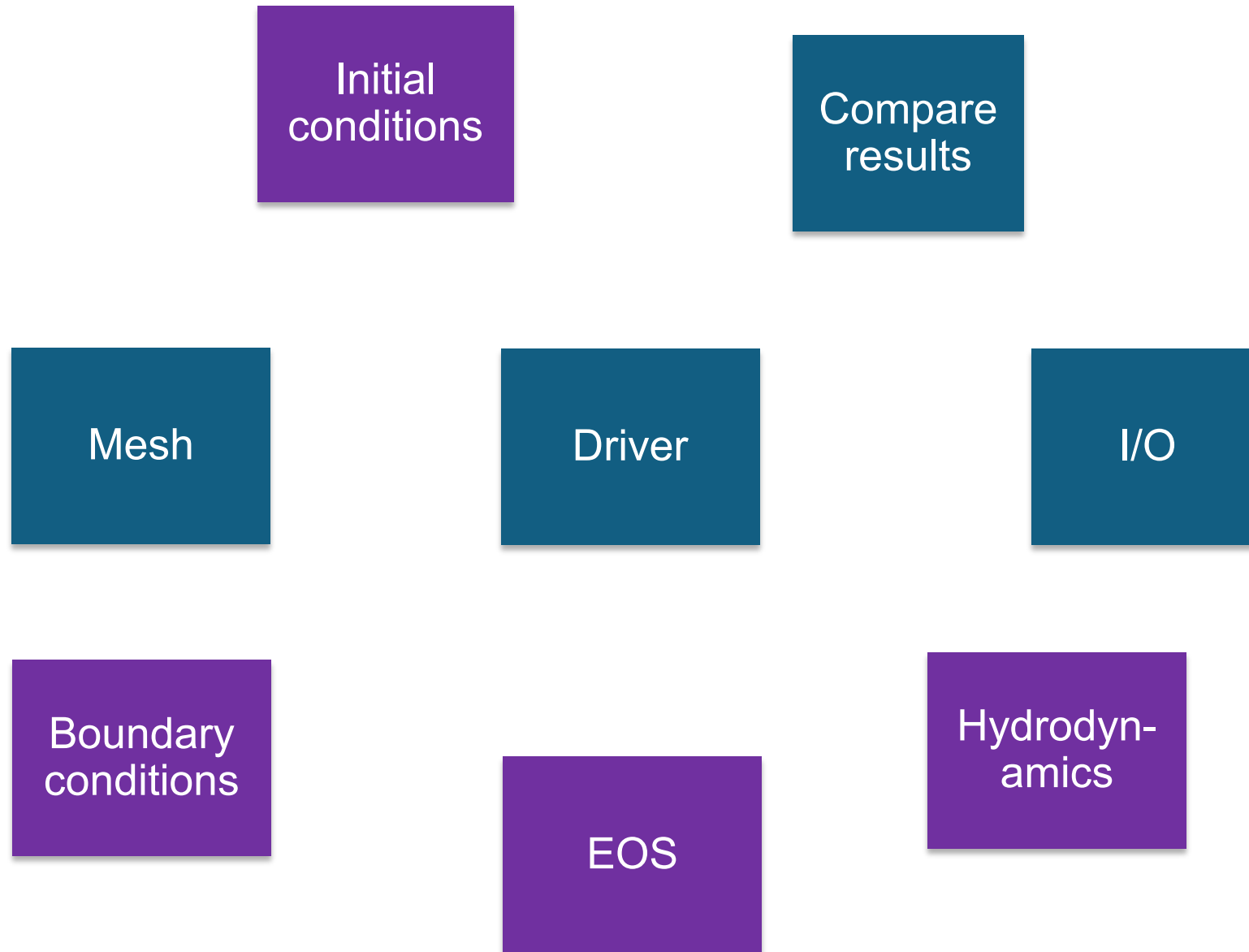
☐ Components evolving with research – physics solvers

- ☐ Initial and boundary conditions
- ☐ Hydrodynamics
- ☐ EOS

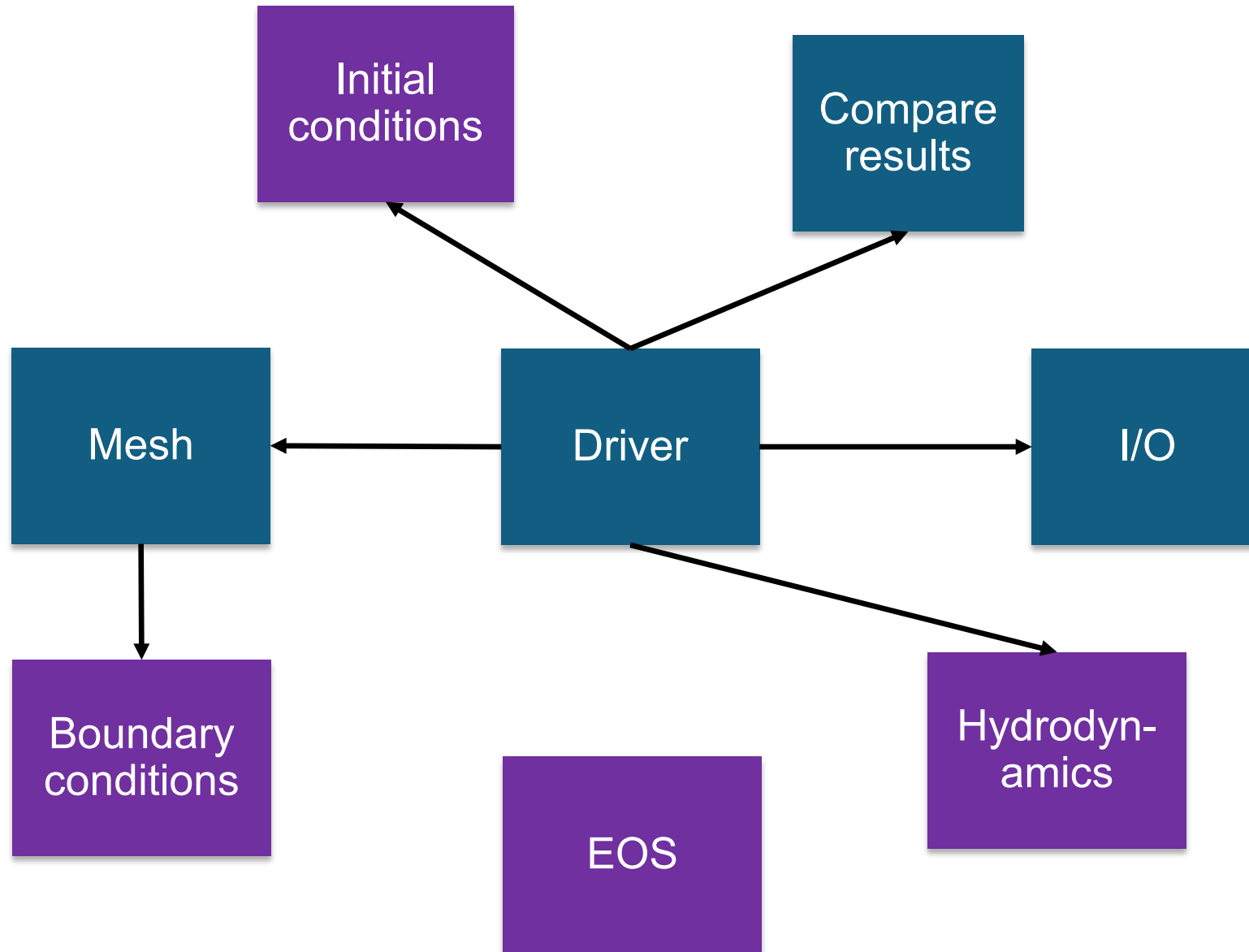
Deeper Dive into some Components

- Driver
 - Iterate over blocks
 - Implement connectivity
- Mesh
 - Data containers
 - Halo cell fill, including application of boundary conditions
 - Reconciliation of quantities at fine-coarse block boundaries
 - Remesh when refinement patterns change
- I/O
 - Getting runtime parameters and possibly initial conditions
 - Writing checkpoint and analysis data

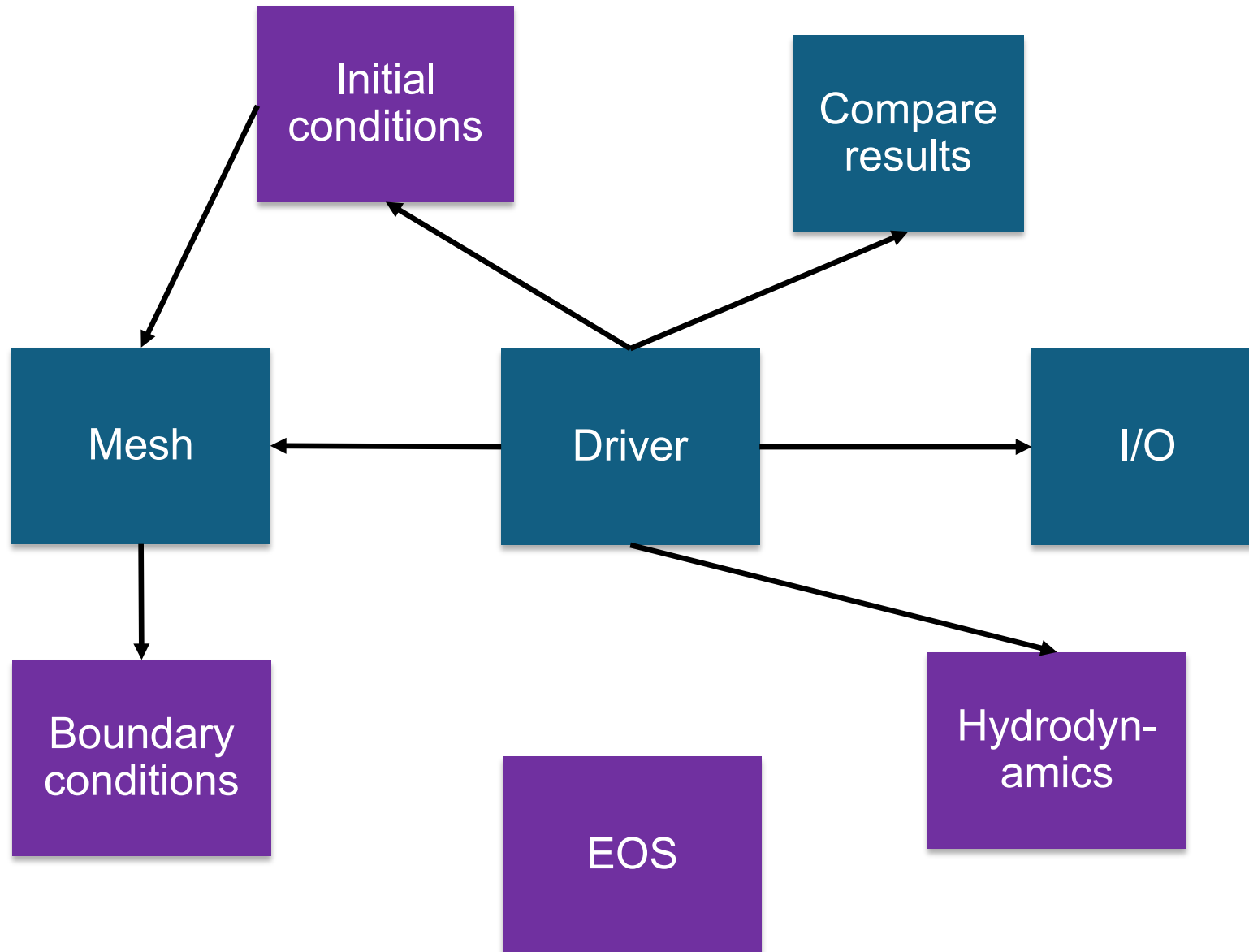
Connectivity



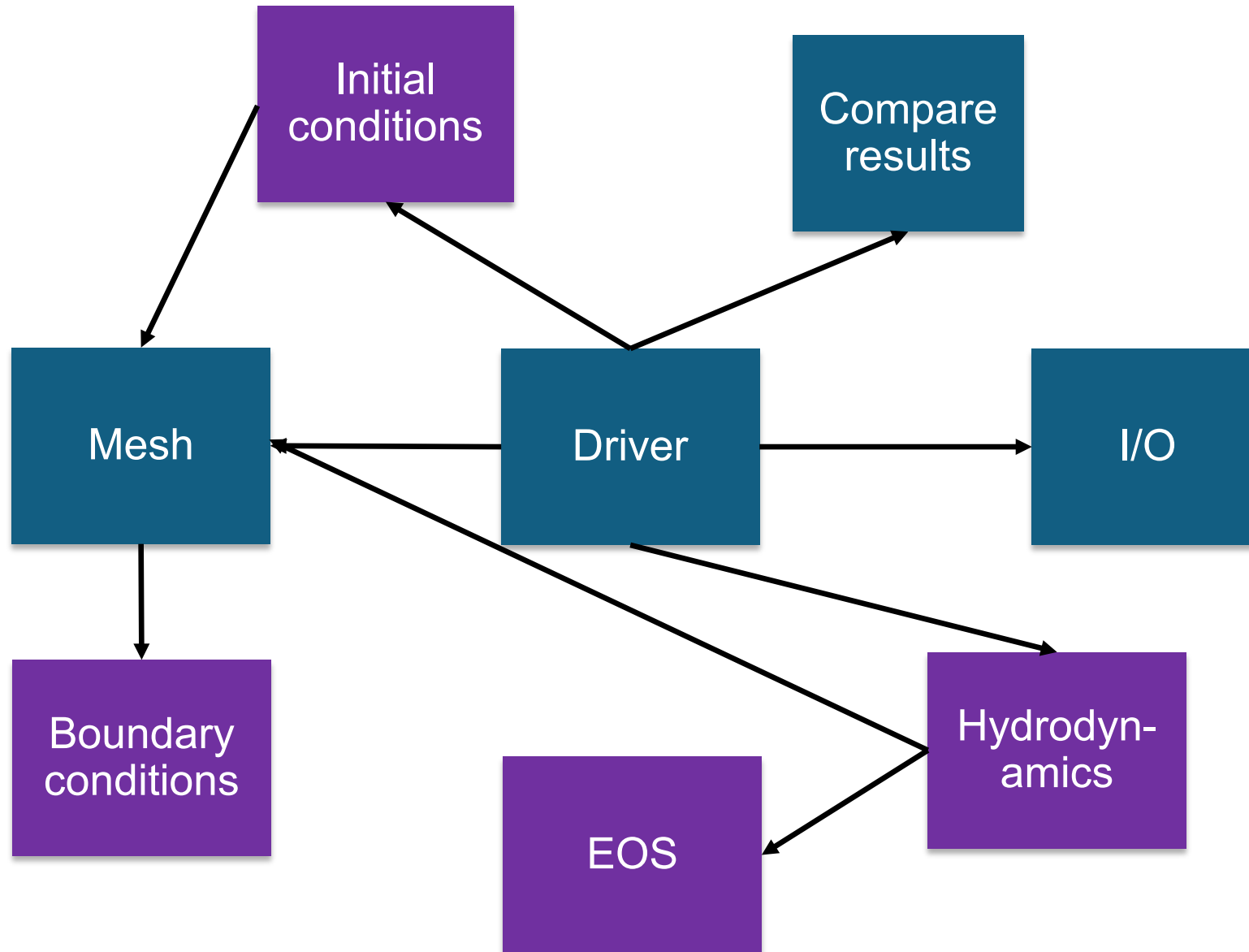
Connectivity



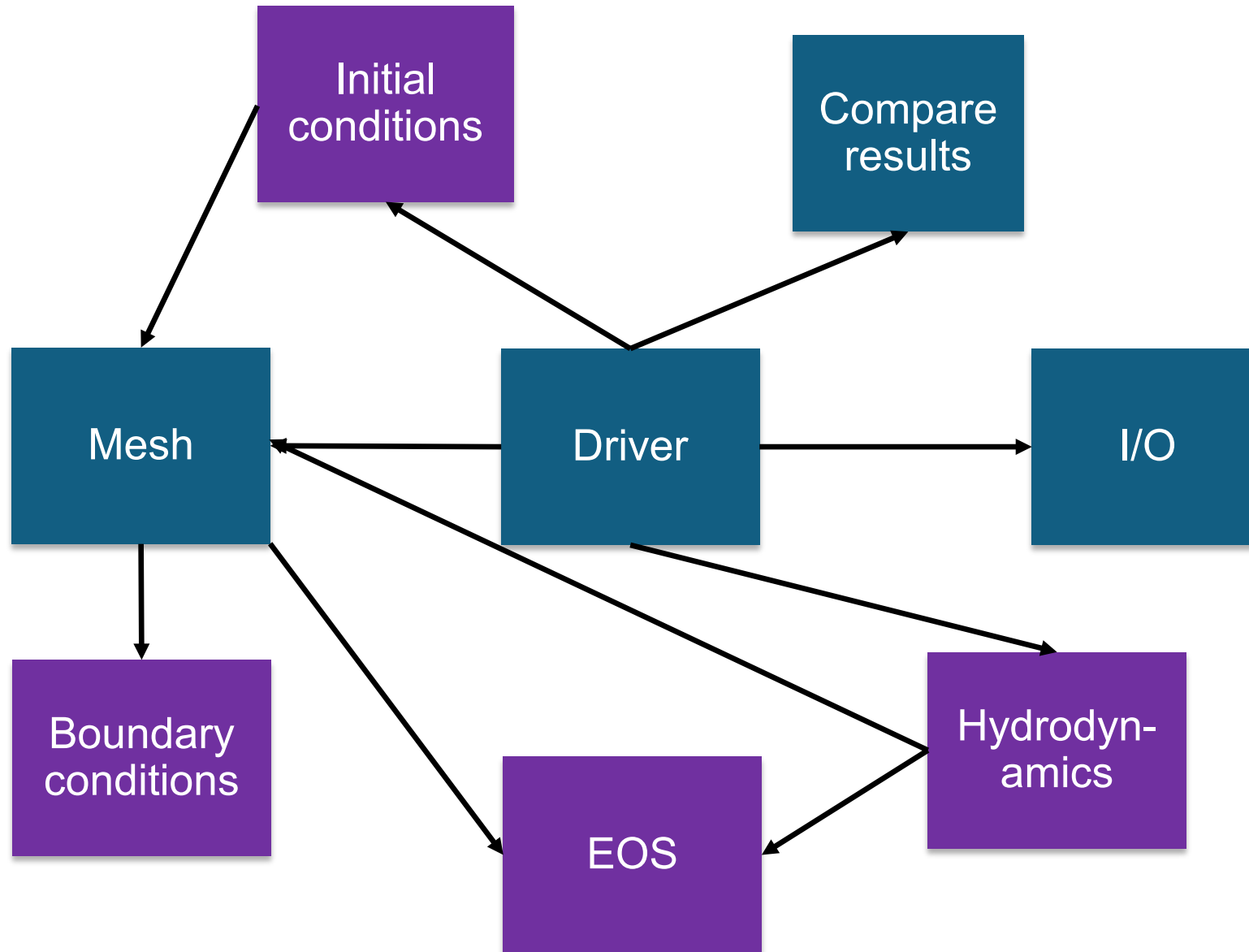
Connectivity



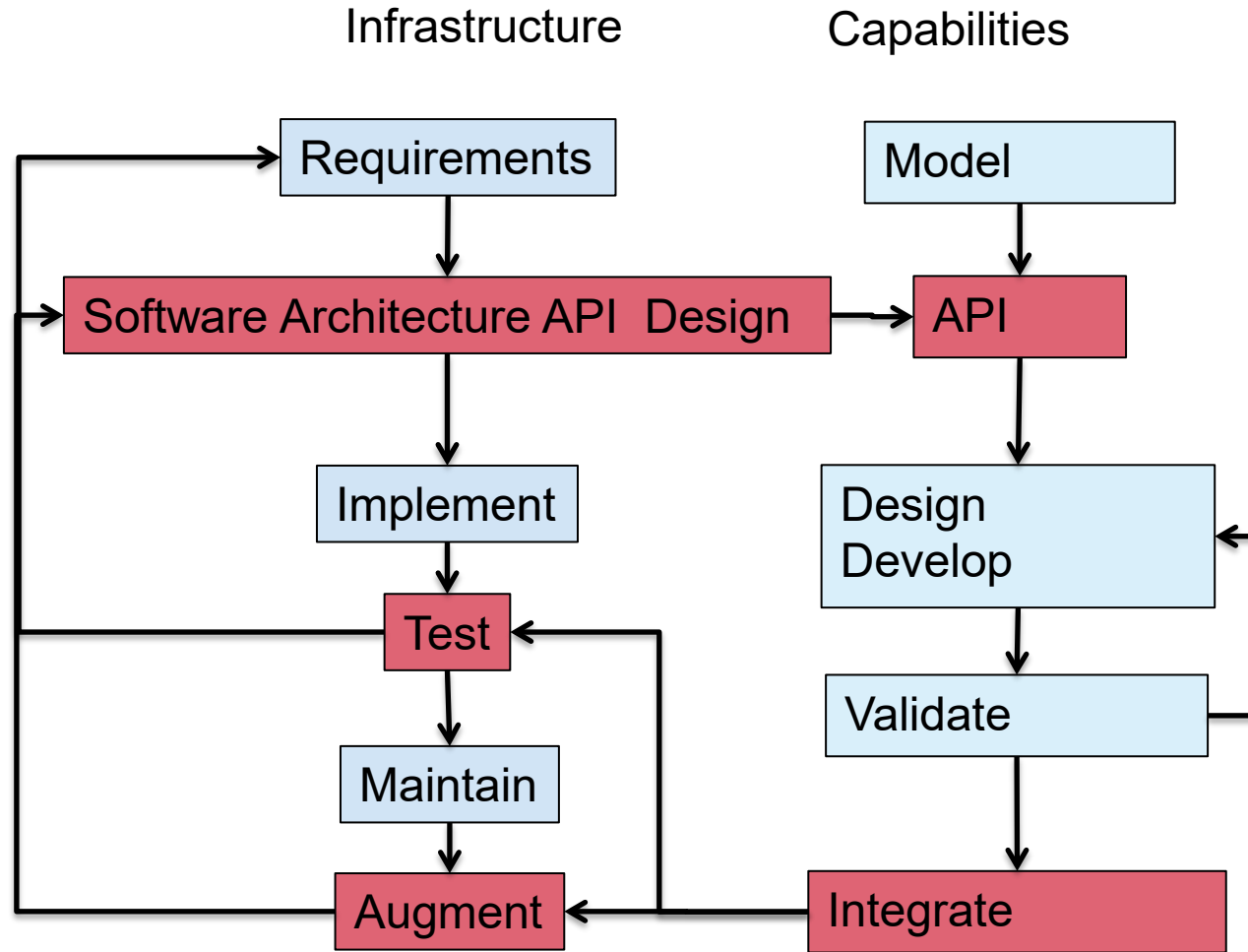
Connectivity



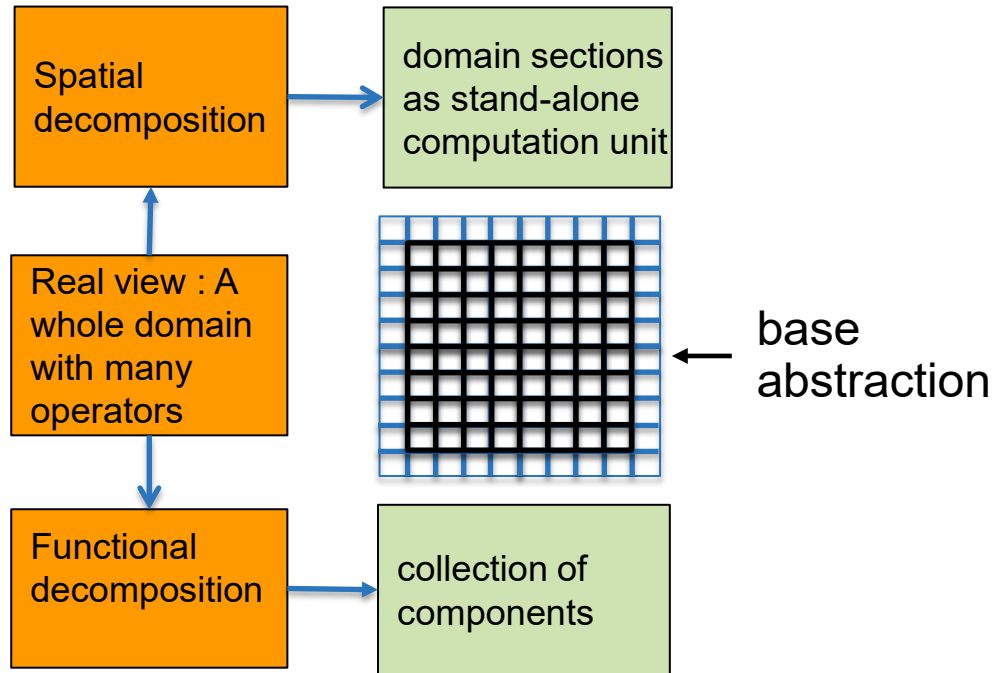
Connectivity



A Design Model for Separation of Concerns



Exploring Design Space – Abstractions



Constraints

- Only infrastructure components have global view
- All physics solvers have block view only

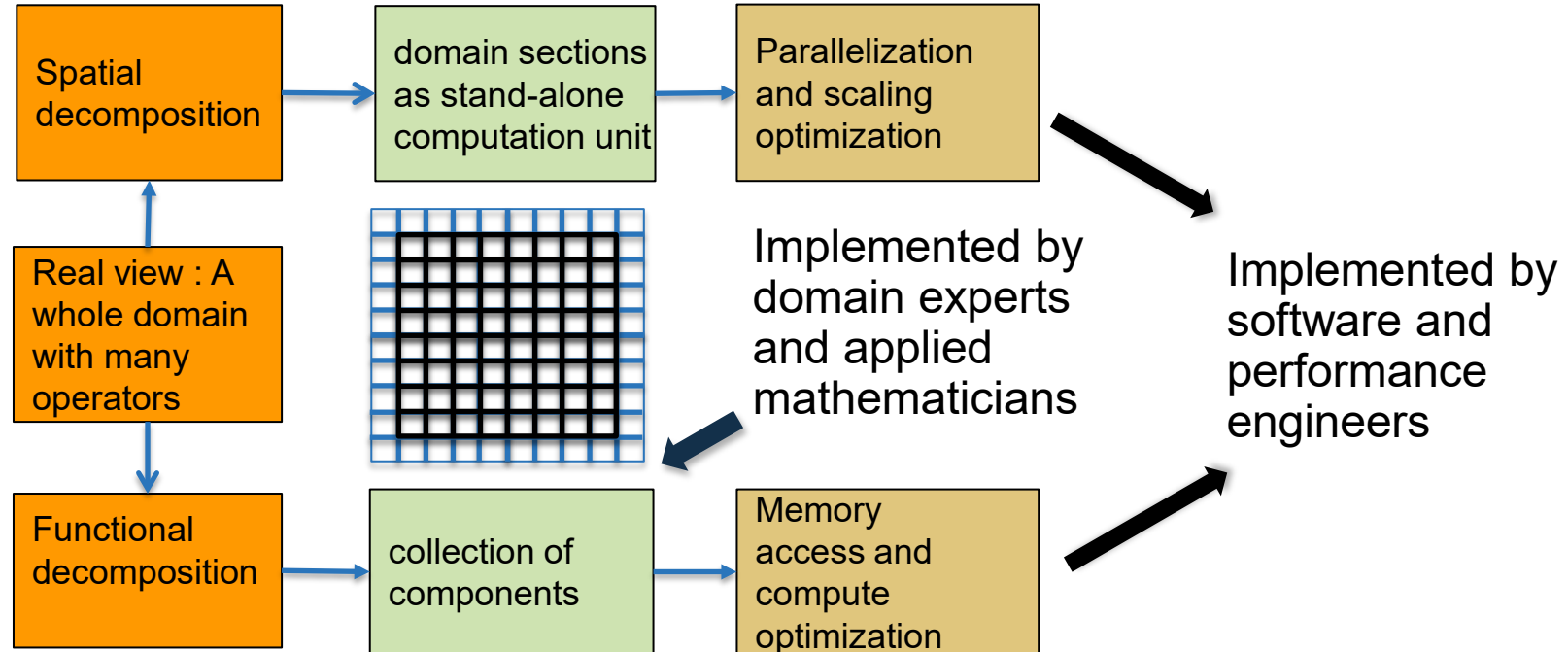
Other Design Considerations

- Data scoping
- Interfaces in the API

Minimal Mesh API

- Initialize_mesh
- Halo_fill
- Access_to_data_containers
- Reconcile_fluxes
- Regrid

Separation of Concerns Applied



Takeaways so Far

- Differentiate between slow changing and fast changing components of your code
- Understand the requirements of your infrastructure
- Implement separation of concerns
- Design with portability, extensibility, reproducibility and maintainability in mind

Building a Test-suite

Elements of test development

- For some tests assertions will suffice
- For others you will need to compare the output against baselines
 - Building a comparison utility is extremely useful
- Also useful to develop diagnostics – indirect ways of verifying behavior
 - Conservation of physical quantities
 - No non-physical values

Building a Test-suite

Elements of test development

- For some tests assertions will suffice
- For others you will need to compare the output against baselines
 - Building a comparison utility is extremely useful
- Also useful to develop diagnostics – indirect ways of verifying behavior
 - Conservation of physical quantities
 - No non-physical values

Building baselines for comparison

- From a known analytical solution
- Manufacture a solution
- Visualize and inspect output and anoint as baseline
- Run a test case up to point A and drop a checkpoint. Run another test case up to a later point B.
 - Use point A to restart and B as the anointed baseline

Building a Test-suite

Elements of test development

- For some tests assertions will suffice
- For others you will need to compare the output against baselines
 - Building a comparison utility is extremely useful
- Also useful to develop diagnostics – indirect ways of verifying behavior
 - Conservation of physical quantities
 - No non-physical values

Building baselines for comparison

- From a known analytical solution
- Manufacture a solution
- Visualize and inspect output and anoint as baseline
- Run a test case up to point A and drop a checkpoint. Run another test case up to a later point B.
 - Use point A to restart and B as the anointed baseline

Apply scaffolding
for selection of
tests ...
explained next

Example – Back to Sedov Blast

Components needed

- Mesh
- Hydrodynamics solver
- Equation of state
- Parallelization

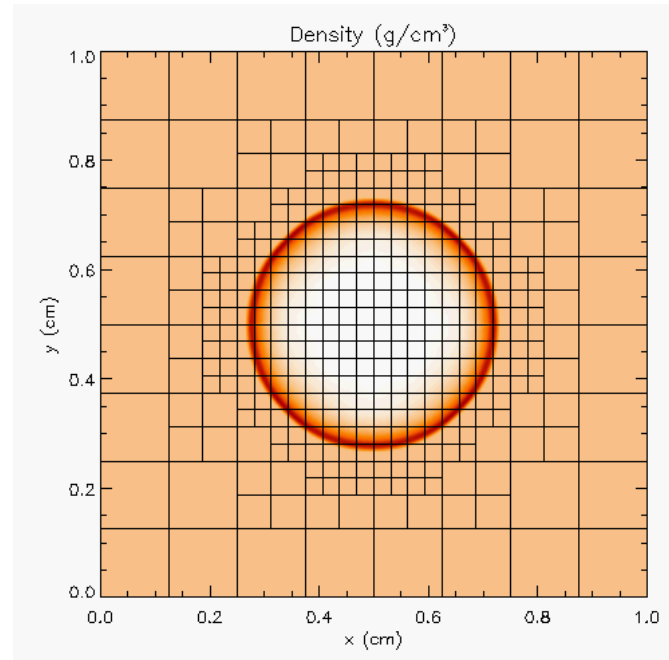
Strategy for development
Think of an application with
analytical solution

Example – Shock Hydrodynamics with Adaptive Mesh Refinement

Components needed

- Mesh
- Hydrodynamics solver
- Equation of state
- Parallelization

Strategy for development
Think of an application with
analytical solution



- Sedov blast wave
- High pressure at the center
- Shock moves out in a circle
- Analytical solution for how far the shock has travelled

Step 1 – Equation of State

- Initialize density and internal energy with known values
- Compute pressure and temperature using EOS
- Next use density and computed pressure as input and compute internal energy and temperature using EOS
- Compare computed values against initialized values

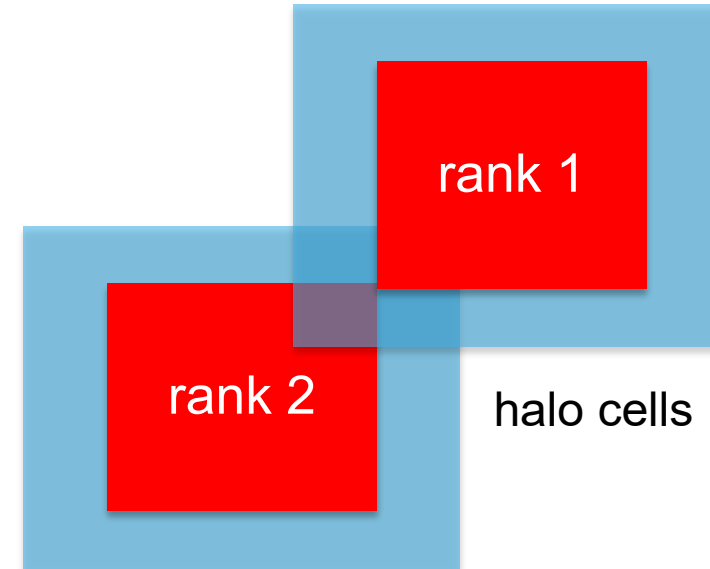
Step 1 – Equation of State

- Initialize density and internal energy with known values
- Compute pressure and temperature using EOS
- Next use density and computed pressure as input and compute internal energy and temperature using EOS
- Compare computed values against initialized values

We have a unit test

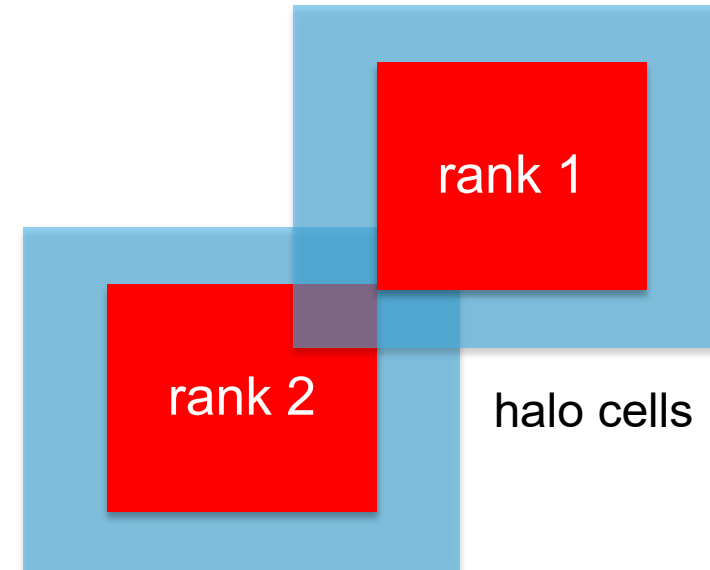
Step 2 – Mesh

- Start with uniform grid
- Domain decomposition for parallelization
 - Halo fill operation
- Initialize the interior (red) with a known function
- Apply halo fill
- Compute values for the halo using the known function
- Compare against filled values



Step 2 – Mesh

- Start with uniform grid
- Domain decomposition for parallelization
 - Halo fill operation
- Initialize the interior (red) with a known function
- Apply halo fill
- Compute values for the halo using the known function
- Compare against filled values



We have another unit test

Step 3 – Hydrodynamics

- Apply initial conditions to the mesh
 - zeroes everywhere except at the center
- Write code for the analytical expression of the distance traveled by the shock
- Do time integration
- At time T compare evolved solution against analytical solution

If both mesh and EOS unit test pass, then any failure is in Hydrodynamics
This is a composite unit test

This is also the idea behind scaffolding

Step 4 – AMR

- The same halo fill unit test for mesh also works for AMR
- Additional functionalities to test are:
 - Fine-coarse boundary resolution
 - Regridding
- Steps in testing
 - Run Sedov with UG
 - Run Sedov with AMR, but no dynamic refinement
 - If failed fault is in flux correction
 - Run Sedov with AMR and dynamic refinement
 - If failed fault is in regridding

Step 4 – AMR

- The same halo fill unit test for mesh also works for AMR
- Additional functionalities to test are:
 - Fine-coarse boundary resolution
 - Regridding
- Steps in testing
 - Run Sedov with UG
 - Run Sedov with AMR, but no dynamic refinement
 - If failed fault is in flux correction
 - Run Sedov with AMR and dynamic refinement
 - If failed fault is in regridding

We have continued to build scaffolding and are using granular testing to pinpoint the cause of error

Step 4 – AMR

- The same halo fill unit test for mesh also works for AMR
- Additional functionalities to test are:
 - Fine-coarse boundary resolution
 - Regridding
- Steps in testing
 - Run Sedov with UG
 - Run Sedov with AMR, but no dynamic refinement
 - If failed fault is in flux correction
 - Run Sedov with AMR and dynamic refinement
 - If failed fault is in regridding

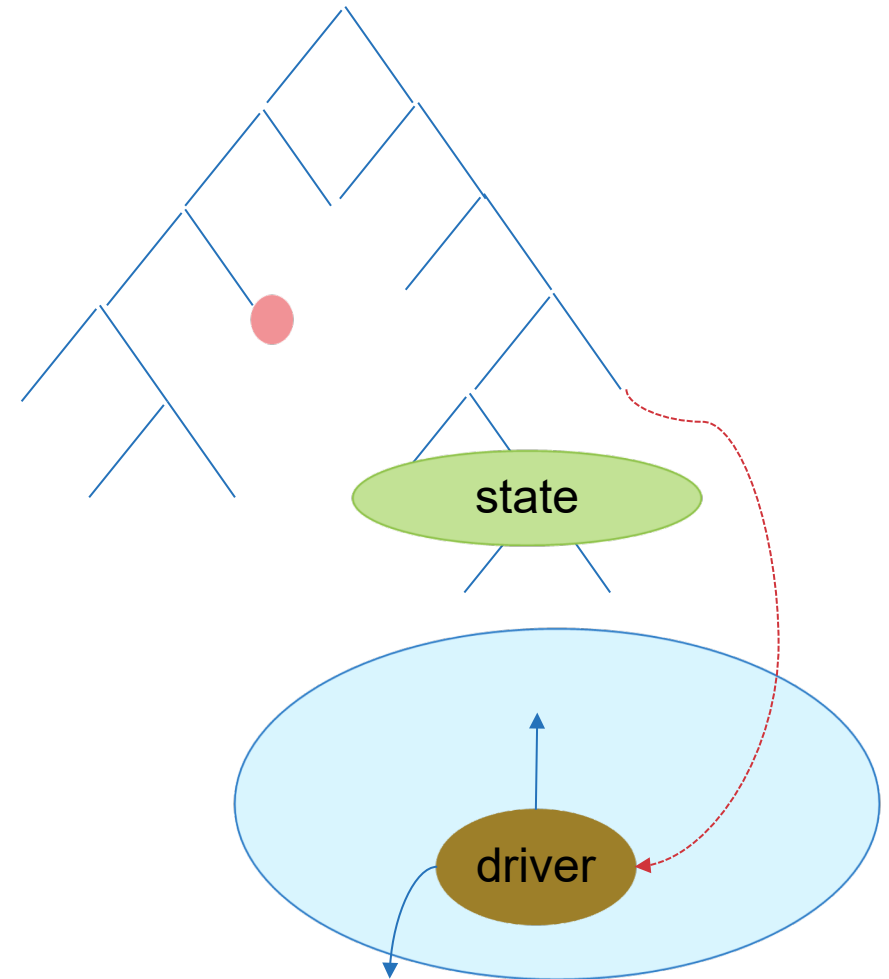
We have continued to build scaffolding and are using granular testing to pinpoint the cause of error

All of these are examples of clear box testing

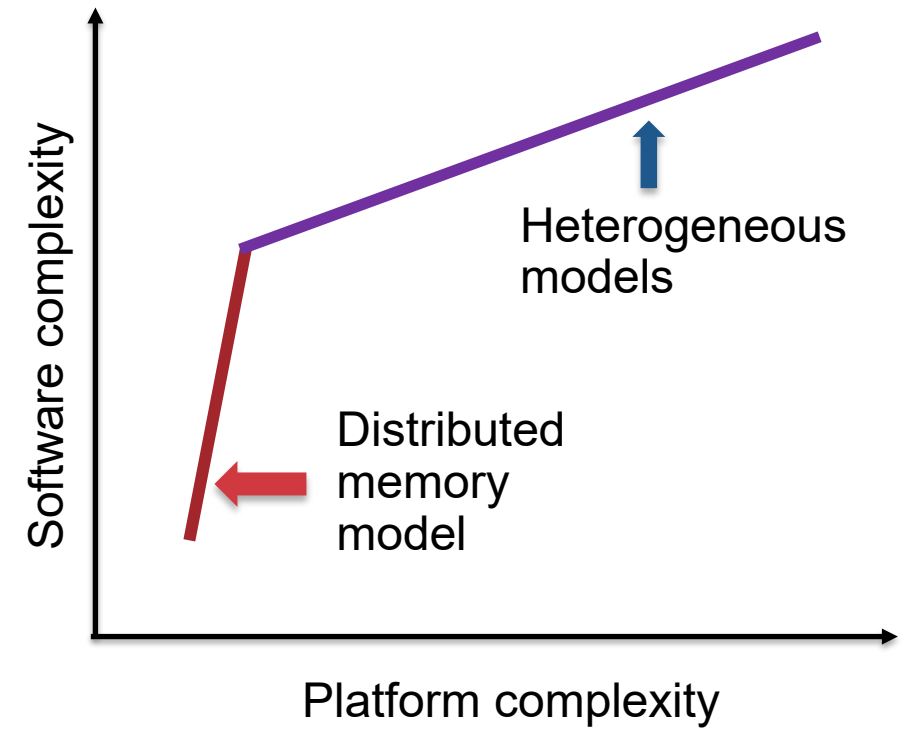
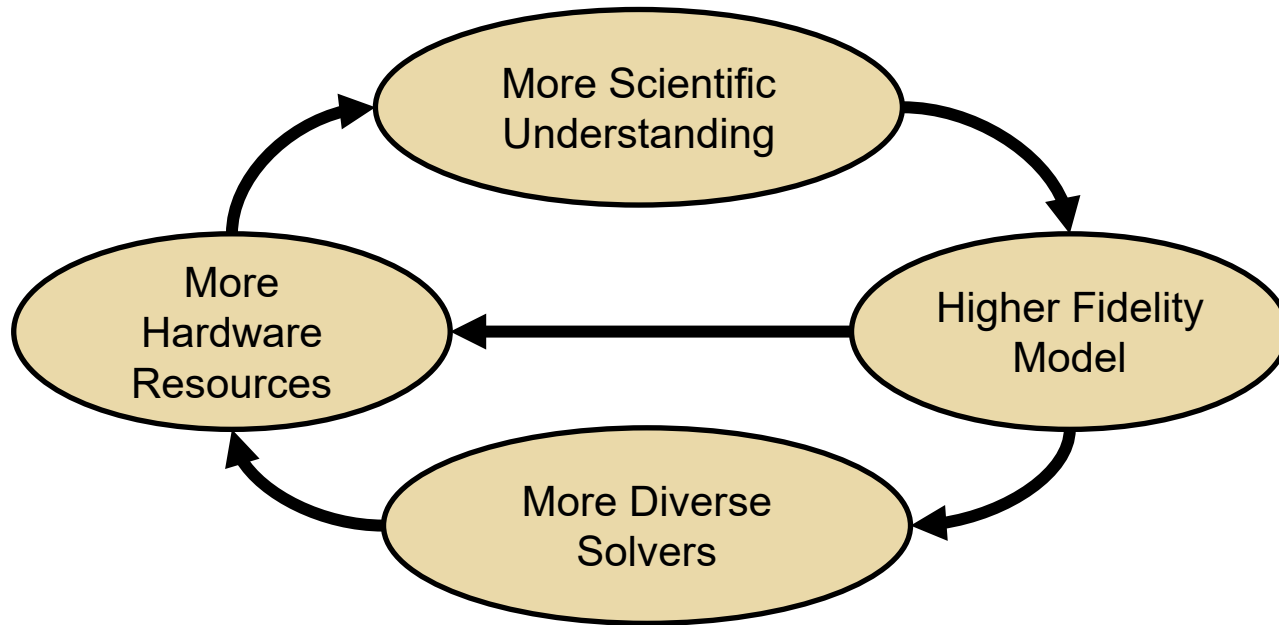
Mixed Clear/Opaque Box Testing For a Legacy Code

There may not be existing tests

- Isolate a small area of the code
- Dump a useful state snapshot
- Build a test driver
 - Start with only the files in the area
 - Link in dependencies
 - Copy if any customizations needed
- Read in the state snapshot
- Restart from the saved state
- Verify correctness
 - Always inject errors to verify that the test is working



New Paradigm in Design Because of Platform Heterogeneity



Mechanisms Needed by the Code

Mechanisms to unify expression of computation

- Minimize maintained variants of source suitable for all computational devices
- Reconcile differences in data structures

Mechanisms Needed by the Code

Mechanisms to unify expression of computation

- Minimize maintained variants of source suitable for all computational devices
- Reconcile differences in data structures

Mechanisms to move work and data to computational targets

- Moving between devices
 - Launching work at the destination
 - Hiding latency of movement
- Moving data off node

Mechanisms Needed by the Code

Mechanisms to unify expression of computation

- Minimize maintained variants of source suitable for all computational devices
- Reconcile differences in data structures

Mechanisms to move work and data to computational targets

- Moving between devices
 - Launching work at the destination
 - Hiding latency of movement
- Moving data off node

Mechanisms to map work to computational targets

- Figuring out the map
 - Expression of dependencies
 - Cost models
- Expressing the map

Mechanisms Needed by the Code

Mechanisms to unify expression of computation

- Minimize maintained variants of source suitable for all computational devices
- Reconcile differences in data structures

Mechanisms to move work and data to computational targets

- Moving between devices
 - Launching work at the destination
 - Hiding latency of movement
- Moving data off node

Mechanisms to map work to computational targets

- Figuring out the map
 - Expression of dependencies
 - Cost models
- Expressing the map

So, what do we need?

- Abstractions layers
- Code transformation tools
- Data movement orchestrators

Underlying Ideas: Unification of Computational Expressions

Make the same code work on different devices

Same algorithm different data layouts or operation sequence:

- A way to let compiler know that "this" expression can be specialized in many ways
- Definition of specializations
- Often done with template meta-programming

More challenging if algorithms need to be fundamentally different

- Support for alternatives

Underlying Ideas: Moving Work and Data to the Target

Parallelization Models

Hierarchy in domain decomposition

- Distributed memory model at node level – still very prevalent, likely to remain so for a while
- Also done with PGAS models – shared with locality being important

Assigning work within the node

- “Parallel For” or directives with unified memory
- Directives or specific programming model for explicit data movement

More complex data orchestration system for asynchronous computation

- Task based work distribution

Underlying Ideas: Mapping Work to Targets

This is how many abstraction layers work

- Infer the structure of the code
- Infer the map between algorithms and devices
- Infer the data movements
- Map computations to devices
- These are specified either through constructs or pragmas

**It can also be the end user who figures out the mapping
In either case performance depends upon how well the mapping is done**

Mechanisms Needed by the Code: Example Flash-X

Mechanisms to unify expression of computation

Macros with inheritance

Mechanisms to move work and data to computational targets

Domain specific runtime

Mechanisms to map work to computational targets

DSL for recipes with code generator

Composability in the source
A toolset of each mechanism
Independent tool sets

Mechanisms Needed by the Code

Mechanisms to unify expression of computation

- Minimize maintained variants of source suitable for all computational devices
- Reconcile differences in data structures

Express code with embedded macros

- Let macros have multiple alternative definitions
- Implement mechanism to select specific macro definition
- Implement mechanism to safely include more than one definition
- Allow inline, recursion and arguments in macros

Code Expressed with macros

```
@M declare
@M directive1
@M loop_2d
.... computation 1
.... computation 2
@M endloop_2d_spl
@M directive2
@M loop_2d_spl
.... computation 3
@M endloop_2d
```

Definitions for CPU

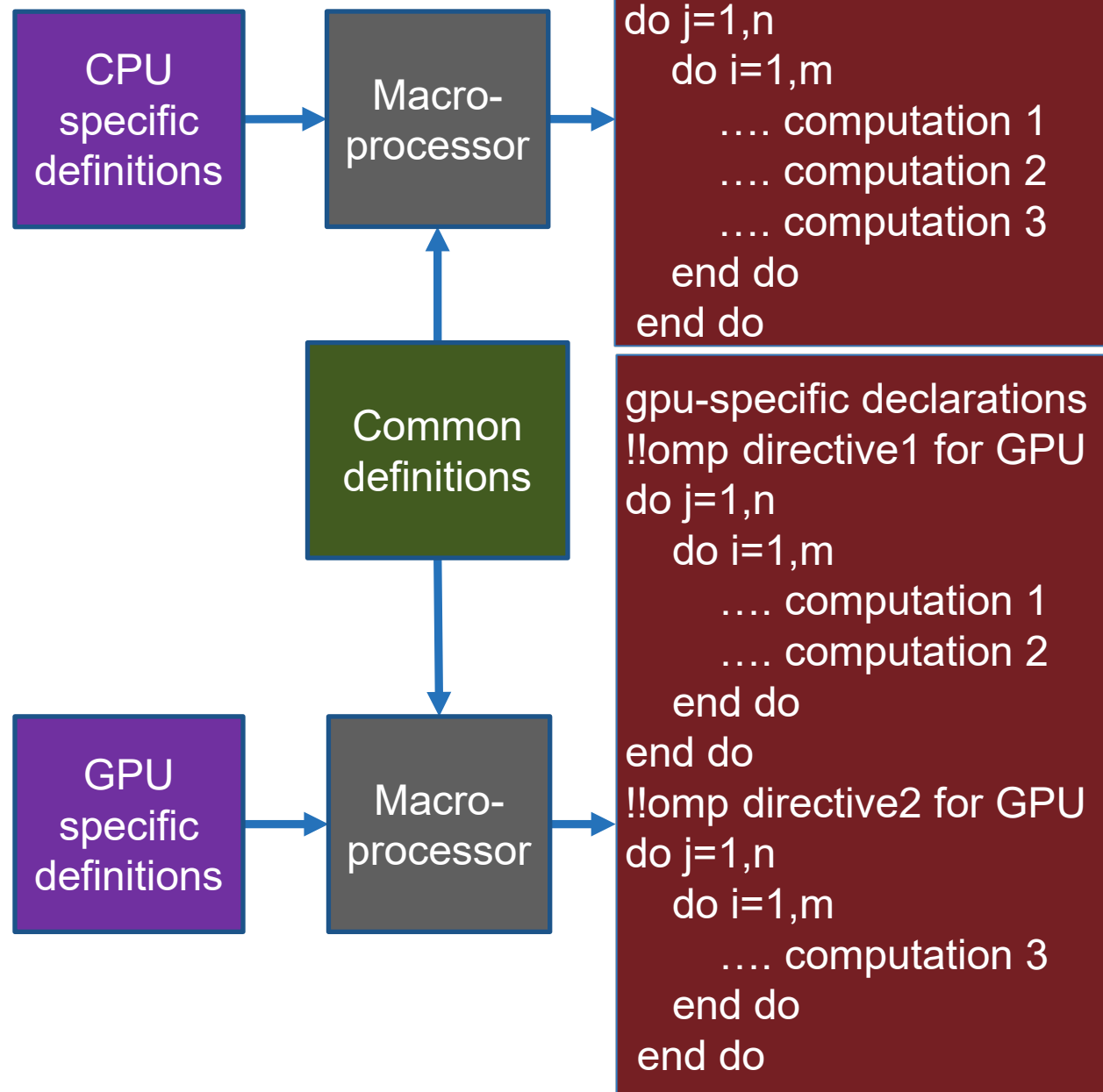
```
[declare]
definition=
cpu-specific declarations
[directive1]
definition=
!!omp directive for cpu
[endloop_2d_spl]
definition=
[directive2]
definition=
[loop_2d_spl]
75definition=
```

Common definitions

```
[loop_2d]
[endloop_2d]
definition=      definition=
do j=1,n          end do
do i=1,m          end do
```

Definitions for GPU

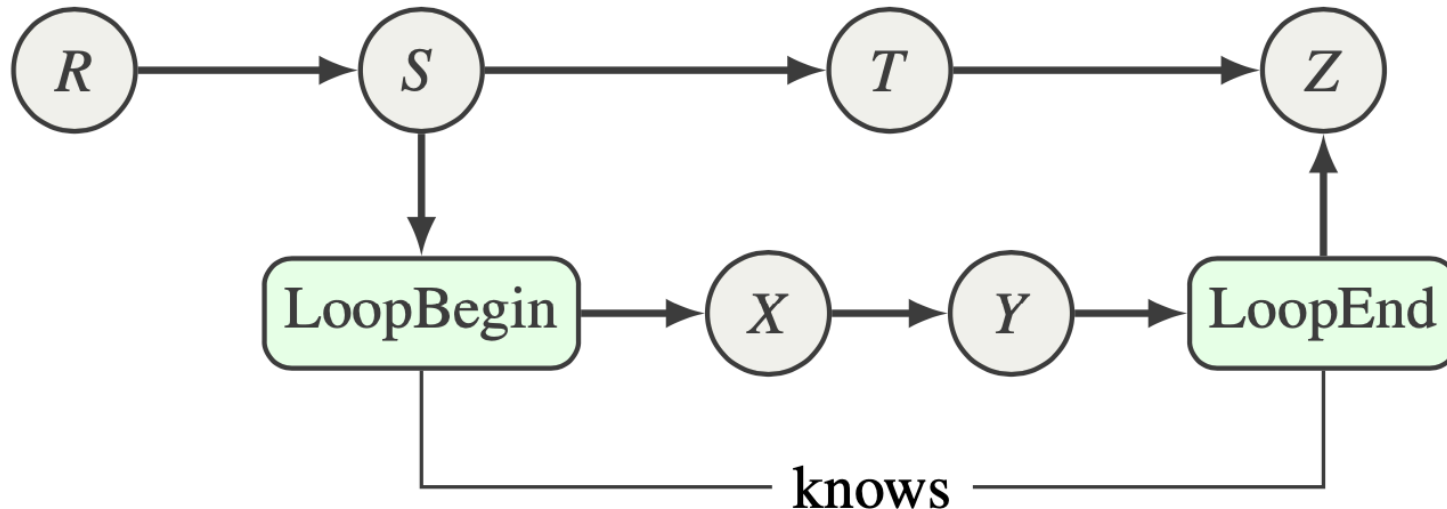
```
[declare]
definition=
gpu-specific declarations
[directive1]
definition=
!!omp directive1 for gpu
[endloop_2d_spl]
definition=
@M endloop_2d
[directive2]
definition=
!!omp directive2 for gpu
[loop_2d_spl]
definition=
@M loop_2d
```



Mechanisms Needed by the Code

Mechanisms to map work to computational targets

- Figuring out the map
 - Expression of dependencies
 - Cost models
- Expressing the map



- CG-Kit generates Control Flow Graph by reading user input (recipe).
 - Represented as a directed acyclic graph (DAG)
 - Each node represents user-defined code generation operations.
 - PST
 - Each edge describes dependencies between nodes.

Mechanisms Needed by the Code

Mechanisms to move work and data to computational targets

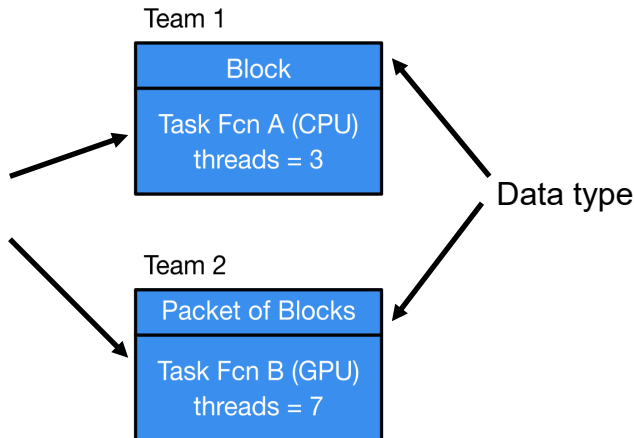
- Moving between devices
 - Launching work at the destination
 - Hiding latency of movement
- Moving data off-node

Milhoja: A Domain Specific Runtime
Capitalizes on the control flow remaining the same from one timestep to next – static analysis during application configuration

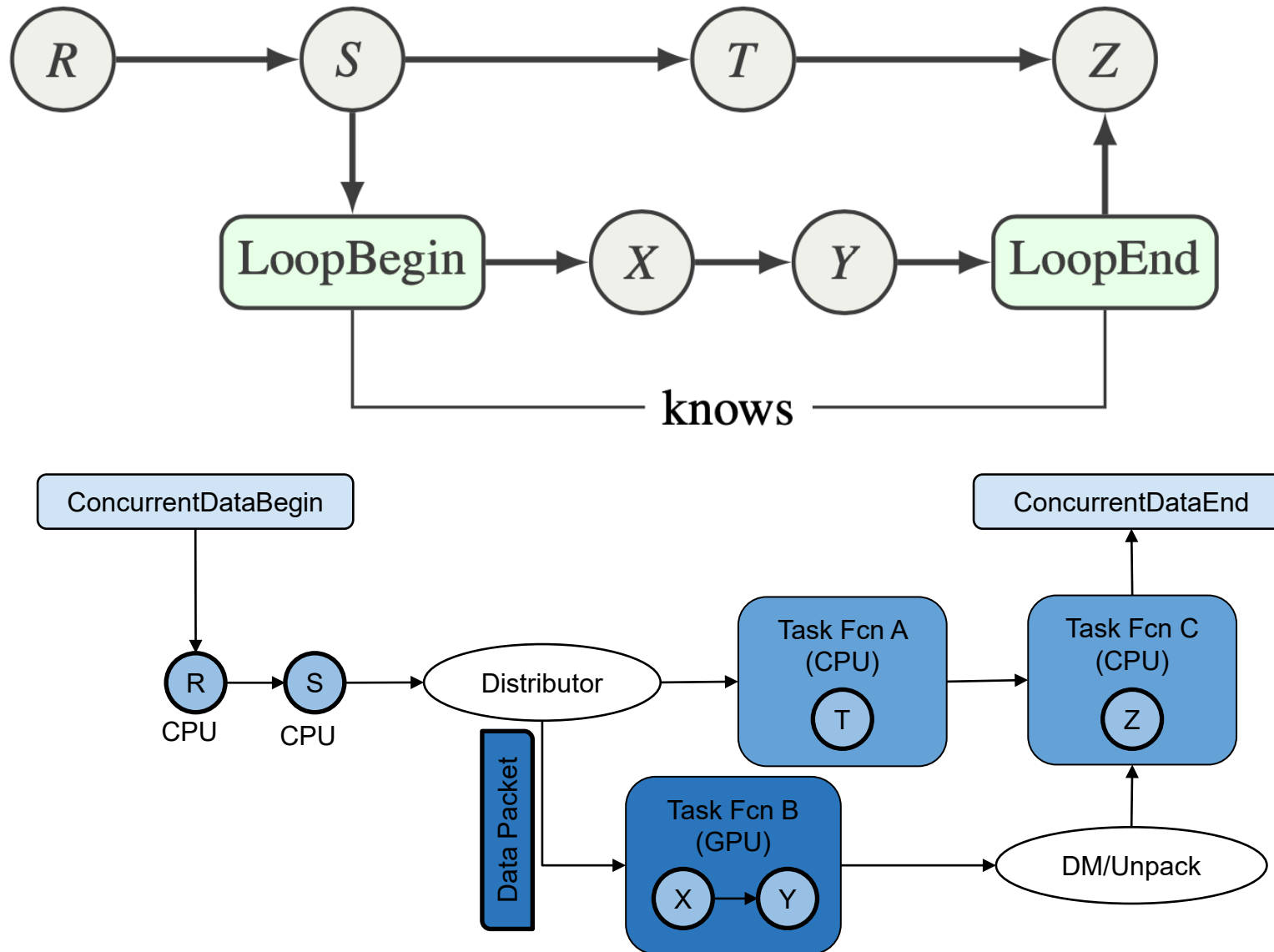
Distributors

- Use block iterator
- Aggregate blocks if necessary
- Initiate asynchronous transfers if necessary
- Push blocks to other elements

Computation to apply to each data item



Orchestration of Data and Computation



Orthogonal Axes of Challenges and Optimization

- ❑ Have a method for expressing algorithmic variants
 - ❑ Without delving into the details of the arithmetic
- ❑ Example -- Flash-X supports two block-structured AMR grid backends
 - ❑ Paramesh: Octree-based, AMReX: Level-based
- ❑ Each has different preferences for flux correction at fine-coarse boundaries
- ❑ For higher order RK integration Communication avoidance – telescoping mode

```
do all_blocks
  ! hydrodynamics updates
end do
call communicate_fluxes() ! p2p communication
do all_blocks
  ! flux correction
end do
```

```
do stage = 1, max_stage
  call fill_guardcells() ! p2p communication
  do all_blocks
    ! block initializations
    ! intra stage calculations
  end do
end do
```

```
do lev = max_level, 1, -1
  call communicate_fluxes() ! p2p communication
  do blocks_on(level = lev)
    ! hydrodynamics updates
    ! flux correction
  end do
end do
```

```
call fill_guardcells() ! p2p communication
do all_blocks
  ! block initializations
  do stage = 1, max_stage
    ! intra stage calculations
  end do
end do
```

Final Takeaways

- Requirements gathering and intentional design are indispensable for sustainable software development
- Many books and online resources available for good design principles
- Research software poses additional constraints on design and verification because of its exploratory nature
 - Scientific research software has further challenges
 - High performance computing research software has even more challenges
 - That are further exacerbated by the ubiquity of accelerators in platforms
- Separation of concerns at various granularities, and abstractions enable sustainable software design and enable verification
- Using good software engineering enables relying on LLM for automating aspects of coding.